Industry Classification

Code explanations, measured results, and oral-exam preparation

21 October 2026

Contents

How to use this guide 2

Code block 01: Imports and resource controls 3

Code block 02: Financial cleaning, splitting, and encoding 4

Code block 03: PCA and classifier fitting 6

Code block 04: Classifier comparison and error analysis 8

Code block 05: Financial distributions and fitting references 12

Code block 06: The score and statistical confidence inputs 15

Code block 07: Learning correctness confidence 17

Code block 08: Evaluation, calibration, and review 18

Code block 09: Inference and limits of representation novelty 21

Worked predictions and calculations 22

Oral-exam questions and answers 25

Reproduction and development evidence 35

Complete per-industry results 37

Complete annotated code 39

Sources and reading 44

How to use this guide

Read this alongside `industry_classification.ipynb`. Comments such as **[06.24]** identify actual statements in the notebook. The first number identifies a code block; the second identifies a statement within it. The numbered catalogues explain every one of the 145 referenced statements. The full annotated code appears at the end. The separate question section gives 95 oral-exam answers.

The project has two connected prediction problems. First, a classifier chooses one of 25 industries from a supplied business-description embedding. Second, a correctness model estimates whether the chosen industry is right. A financial observation is supporting evidence for this second question. There is no need to know the true industry of a company when making its prediction.

The required course score is

$$S = F_{1,\text{weighted}}(1 - B), \qquad B = \frac{1}{N}\sum_{i=1}^{N}(c_i - z_i)^2, \qquad z_i = \mathbf{1}(\hat{y}_i = y_i).$$

Here c_i is estimated correctness probability. It does not weight the F1 confusion counts. The professor replaced the earlier scoring method and withdrew its 85% target. About 0.66 for plain LR on the private test is an orientation value; final bonus thresholds remain unspecified. Neither this guide nor the notebook claims a private score or a guaranteed bonus.

What was measured

Method	Weighted F1	Brier	Official score	Δ vs plain LR
Plain LR	0.777788	0.137087	0.671163	0.000000
Plain k-NN	0.771401	0.141496	0.662251	-0.008912
Raw vote	0.780881	0.137380	0.673604	0.002441
Text calibration	0.780881	0.134588	0.675785	0.004621
Financial calibration	0.780881	0.133534	0.676607	0.005444

The selected model's local weighted F1 is **0.780881**, and its official score is **0.676607**. Plain LR on these exact same validation companies scores **0.671163**, giving a point difference of **0.005444**, or **0.544 percentage points**. This is the valid local baseline comparison. Comparing a local score directly with the professor's private baseline would mix datasets.

The improvement is modest. A paired approximate bootstrap grouped by identical validation embedding strings gives a 95% interval **[-0.004145, 0.015126]** for the score difference. It includes zero. Thus the result is higher on this split, but a reliably positive advantage across populations is not established. That limitation is part of the result, rather than a reason to substitute a different metric or claim the unknown threshold has been reached.

All **9 code blocks** ran in a fresh Jupyter kernel without execution errors, producing **7 scientific figures**. The complete cloud run, including separate interface checks, took **80.5 seconds**. A conservative runner-plus-kernel peak-memory upper bound was **1.37 GiB**. These measurements are comfortably below 45 minutes and 16 GiB on this machine; they do not certify timings on every mid-tier laptop.

The executable core has **145 Python statements**, compared with 208 in the earlier full notebook when comments and docstrings are excluded. Statements are not a measure of runtime. Short functions, a shared classifier-analysis loop, and a fixed retained configuration reduce repetition. Additional comments deliberately increase explanatory text without increasing the algorithm's scope. Discarded model-search code and input datasets are not part of the submitted package.

Requirements and evidence

Requirement	Evidence in the notebook
Supplied company data	31,706 actual records, 25 industries, 768 embedding coordinates; actual scope reported
Text embedding classification	LR on original coordinates; k-NN on fitting-only PCA128 coordinates
Dimensionality reduction and visualization	Fitting-only PCA; labeled two-dimensional scatterplot; retained variance
Compare required approaches	Accuracy, weighted/macro F1, per-industry reports, confusion plots, shared errors
Optional ensemble	Development-selected 25/75 probability vote with aligned class order
Statistical financial evidence	Fitting-only industry ranks from three ratios; bounded compatibility and availability
Missing values and outliers	Median financial-ID aggregation; infinities as missing; rank-bounded extremes; losses retained
Confidence and review	Held-out correctness calibration, Brier, reliability bins, fixed review policy
Output contract	Industry-name and finite [0,1] confidence vectors in input-row order
Resource budget	Complete fresh-kernel execution measured; no very large neural network
Pinned dependencies	requirements.txt pins the direct numerical and Jupyter dependencies
Private threshold	Not known or claimed; compare local methods on identical records

The main data contain **31,706 records**, **31,705 unique company IDs**, and 25 industry names. The assignment's 40,000+ description refers to a broader data scope; this project reports the supplied subset. The financial file contains 29,561 records and 29,399 unique IDs. Financial rows are not additional labeled companies to add to the main dataset count.

Code block 01: Imports and resource controls

The numerical work uses NumPy and scikit-learn; pandas handles tables and joins; matplotlib and seaborn provide the required visual analyses. `display` makes tables readable in Jupyter. Import statements make these implementations available; they do not themselves train models or read input files.

`ast.literal_eval` converts the supplied literal coordinate lists. Ordinary `eval` is unnecessary. `perf_counter` measures elapsed time with a monotonic clock, so wall-clock adjustments do not change the elapsed-time calculation. The timer starts after imports. The external full-run time includes kernel startup and verification in addition to the reported notebook timer.

`threadpool_limits(2)` limits numerical-library threads. k-NN also requests two jobs. These choices keep resource use moderate; they do not impose an operating-system memory limit or prove a laptop timing. The requirements file pins direct dependencies, and Python 3.12 was used for the verified run.

Numbered code references

- **[01.01]** Parse the supplied embedding strings as literal Python values.
- **[01.02]** Measure elapsed time with a monotonic clock.
- **[01.03]** Use arrays and vectorized numerical operations.
- **[01.04]** Load, join, group, and summarize tables.

- **[01.05]** Create and label figures.
- **[01.06]** Draw readable statistical plots.
- **[01.07]** Display tables inside Jupyter.
- **[01.08]** Import the principal-component transformation.
- **[01.09]** Import the classifier used for industries and correctness confidence.
- **[01.10]** Import the required classification metrics.
- **[01.11]** Split whole company-ID groups reproducibly.
- **[01.12]** Import the distance-weighted neighbor classifier.
- **[01.13]** Chain confidence scaling and logistic regression into one fitted object.
- **[01.14]** Encode industry names and scale the six confidence features.
- **[01.15]** Control numerical-library thread use.
- **[01.16]** Limit numerical-library work to two threads.
- **[01.17]** Start the notebook's elapsed-time measurement.

Code block 02: Financial cleaning, splitting, and encoding

Financial cleaning and joining

The three ratio columns are `net_profit_margin`, `ebitda_margin`, and `asset_turnover`. Before changing them, the notebook counts both NaN and infinity so the raw data limitations remain visible.

Ratio	Raw NaN	Raw infinity
net profit margin	1111	2131
ebitda margin	19279	824
asset turnover	1363	5

Infinities become NaN. This does not replace them with zero, which would be a valid numerical value with a different economic meaning. Finite negative values remain: a loss is not automatically an error. The group-by-ID median produces at most one financial row per company. It is a transparent robust aggregation where accounting-period metadata are unavailable.

The main table is left-joined to those aggregated financial records. Companies without financial data remain in the prediction population. A many-to-one join check rejects accidental row multiplication. Aggregation uses only a company's supplied financial records, without its industry or cross-company fitted statistics.

Three disjoint roles

Role	Records	Unique IDs	What learns from them
Fitting	25681	25680	PCA, industry classifiers, financial reference distributions
Calibration	2854	2854	Scaling and correctness-confidence coefficients

Role	Records	Unique IDs	What learns from them
Validation	3171	3171	Metrics and diagnostics; no fitted parameters

The first split reserves roughly 10% of IDs for validation. The second holds out 10% of the remaining development IDs for confidence calibration, leaving about 81% of all records for base-model fitting and 9% for calibration. Actual counts differ slightly because group counts are integer and one ID has two distinct main records. All three ID sets were checked to be disjoint.

During development, the final calibration pool was further divided into 1,427 correctness-calibration and 1,427 tuning records. Candidate scores used tuning labels after their confidence models fitted on the other half. After selection, the final confidence model uses all 2,854 calibration records. The final notebook keeps the selected constants rather than repeating the search. It reproduces the retained training and inference method from scratch.

`GroupShuffleSplit` groups by company ID, not by industry or embedding string. It is not stratified. The actual fitting, calibration, and validation subsets were checked to contain all 25 industries. Different companies can still have the same stored representation; block 09 measures that overlap.

Numerical matrices and labels

Each string is parsed to a list, then converted to float32. One matrix row is one company and one column is one embedding coordinate. The fitting matrix has shape **(25,681, 768)**, calibration **(2,854, 768)**, and validation **(3,171, 768)**. Coordinate values were checked to be finite. The supplied vectors are approximately unit norm.

`LabelEncoder` maps fitting industry names to integer codes. The same mapping encodes held-out targets and decodes predictions. These integers are class identifiers rather than ordered quantities. The industry frequency table counts the actual fitting records: Materials has 6,797, while Consumer Staples Distribution & Retail has 151. That approximately 45-fold difference motivates per-industry and macro metrics in addition to weighted F1.

The list comprehensions apply the same conversion to each element of the ordered `parts` list. Tuple unpacking assigns the resulting three matrices or target arrays to the fitting, calibration, and validation variable names. This replaces repeated conversion code while preserving the row order within each subset. A Series index is removed deliberately when constructing arrays; the target and feature conversions both use those same ordered subsets.

Numbered code references

- **[02.01]** Name the three available financial ratios.
- **[02.02]** Load the supplied financial records.
- **[02.03]** Count missing and infinite values separately before cleaning.
- **[02.04]** Treat infinities as missing and aggregate repeated financial IDs by median.
- **[02.05]** Keep every company while joining at most one financial record per ID.
- **[02.06]** Reuse a short helper for company-group splits.
- **[02.07]** Select one seeded split without separating repeated company IDs.
- **[02.08]** Select rows by their integer positions.
- **[02.09]** Reserve about ten percent of company IDs for validation.
- **[02.10]** Hold out part of development data for correctness calibration.

- **[02.11]** Keep one consistent fitting/calibration/validation order.
- **[02.12]** Reuse the same embedding conversion at training and inference.
- **[02.13]** Parse coordinate lists and store the numerical matrix as float32.
- **[02.14]** Convert each subset once and unpack the three matrices.
- **[02.15]** Learn industry-to-code mapping from fitting labels.
- **[02.16]** Apply that same mapping to each target vector.
- **[02.17]** Print the actual record counts instead of assuming exact percentages.
- **[02.18]** Count the records that actually teach the classifiers.
- **[02.19]** Display the required fitting-industry frequency table.

Code block 03: PCA and classifier fitting

Principal components

Write the fitting matrix as X with N_f rows and $d = 768$ columns. PCA learns the fitting mean μ and eigenvectors of its centered covariance matrix:

$$X_c = X - \mathbf{1}\mu^T, \qquad \Sigma = \frac{X_c^T X_c}{N_f - 1}, \qquad \Sigma v_j = \lambda_j v_j.$$

For the retained axes V_m, a new vector is transformed as $z = (x - \mu)^T V_m$. Explained variance for component j is $\lambda_j / \sum_{r=1}^{d} \lambda_r$. The first two explain **25.29%** of fitting variance; the retained 128 explain **87.62%**. These are feature variance percentages, not accuracies.

The notebook fits PCA on fitting embeddings only and then calls the same transformation on fitting and new data. The two-dimensional scatterplot uses the first two coordinates. k-NN uses all 128; LR uses the original 768. The original embedding geometry is retained rather than standardizing every coordinate arbitrarily. PCA still centers the data internally.

The scatterplot shows some separated regions and substantial central overlap. The pharmaceutical region is visible, but a two-dimensional projection cannot show all boundaries or establish inseparability in the original space. Axis signs can reverse across equivalent eigenvector conventions without changing the subspace's meaning. The model comparison supplies the predictive evidence.

Logistic regression

For industry k, the linear score is $a_k(x) = w_k^T x + b_k$, followed by softmax:

$$p_k(x) = \frac{\exp(a_k(x))}{\sum_{r=1}^{K} \exp(a_r(x))}.$$

The classifier minimizes regularized negative log likelihood. Schematically,

$$-\frac{1}{N_f} \sum_i \log p_{y_i}(x_i) + \frac{\lambda}{2} \|W\|_2^2.$$

The exact objective normalization is implementation-dependent; in the library interface C is inversely related to penalty strength. C=10 weakens regularization relative to C=1. The maximum iteration

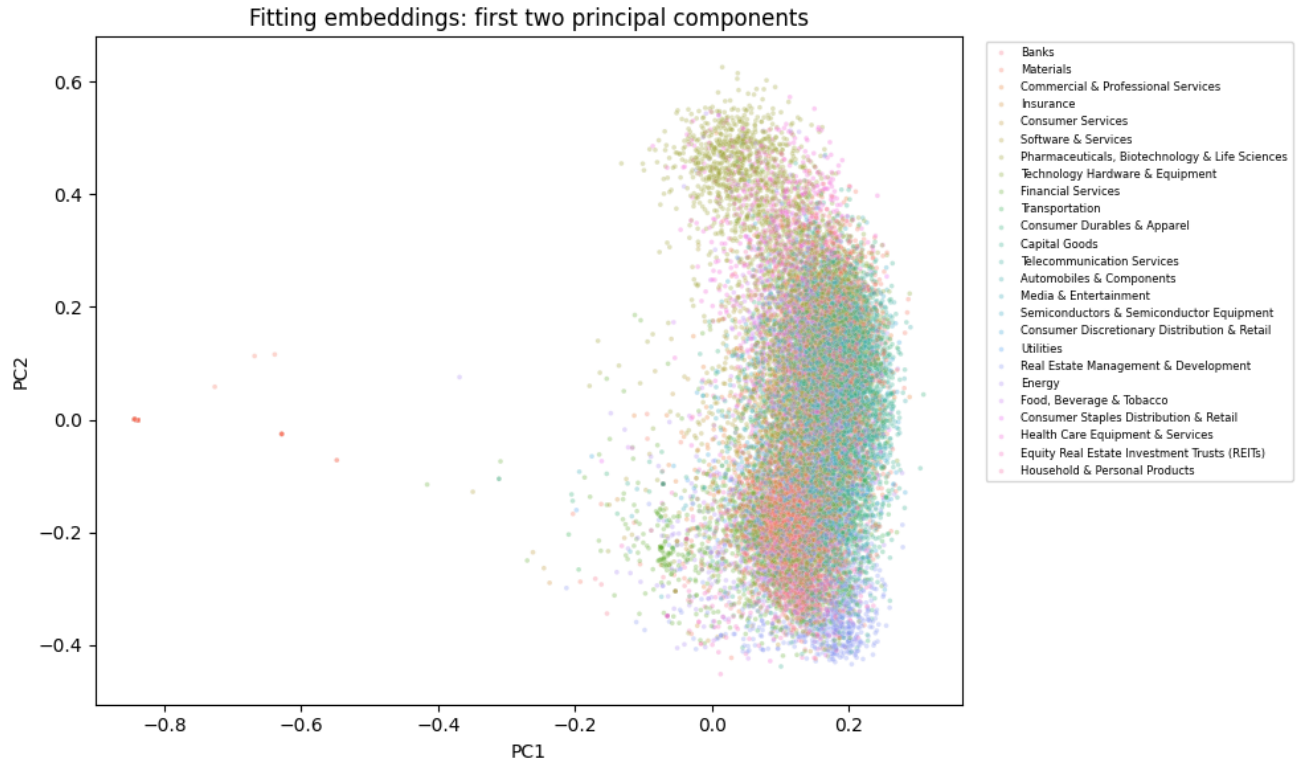


Figure 1: Fitting embeddings projected onto the first two PCA coordinates.

count is a cap, rather than an instruction to perform exactly 1,000 training epochs. This run did not require suppressing a convergence warning.

k-nearest neighbors

k-NN stores fitting vectors and labels. For a query z, Euclidean distance is $d_i = \sqrt{\sum_j (z_j - z_{ij})^2}$. Among its 15 selected neighbors, class k receives normalized inverse-distance weight:

$$p_k^{NN}(z) = \frac{\sum_{i \in \mathcal{N}_{15}(z)} d_i^{-1} \mathbf{1}(y_i = k)}{\sum_{i \in \mathcal{N}_{15}(z)} d_i^{-1}}.$$

Exact zero distances receive the library's special zero-match treatment. It is not implemented as an uncontrolled infinity in this notebook. Fitting is cheap relative to distance queries, so full execution time is the relevant resource report. The helper returns two aligned class-probability matrices and one nearest-distance vector. That distance is a confidence signal, not a third class label.

Numbered code references

- **[03.01]** Learn 128 PCA axes using fitting embeddings only.
- **[03.02]** Project fitting embeddings with the same transformation later used for new inputs.
- **[03.03]** Prepare the first two coordinates and their industry labels for plotting.
- **[03.04]** Choose a figure size that leaves room for the industry legend.
- **[03.05]** Color companies by industry to inspect overlap in two dimensions.

- **[03.06]** Place the many industry names beside the scatterplot.
- **[03.07]** Identify the projected fitting data in the title.
- **[03.08]** Adjust spacing so labels remain visible.
- **[03.09]** Show the PCA scatterplot.
- **[03.10]** Report how much fitting variance the displayed and retained components explain.
- **[03.11]** Fit logistic regression on the original embedding coordinates.
- **[03.12]** Store the fitting PCA vectors and labels for distance-weighted 15-neighbor prediction.
- **[03.13]** Reuse the base-model probabilities and nearest-distance calculation.
- **[03.14]** Transform new vectors with the already fitted PCA.
- **[03.15]** Return aligned LR probabilities, k-NN probabilities, and nearest distances.
- **[03.16]** Compute calibration and validation model inputs once.

Code block 04: Classifier comparison and error analysis

For class k, count true positives TP_k, false positives FP_k, and false negatives FN_k. Then

$$P_k = \frac{TP_k}{TP_k + FP_k}, \quad R_k = \frac{TP_k}{TP_k + FN_k}, \quad F_{1,k} = \frac{2TP_k}{2TP_k + FP_k + FN_k}.$$

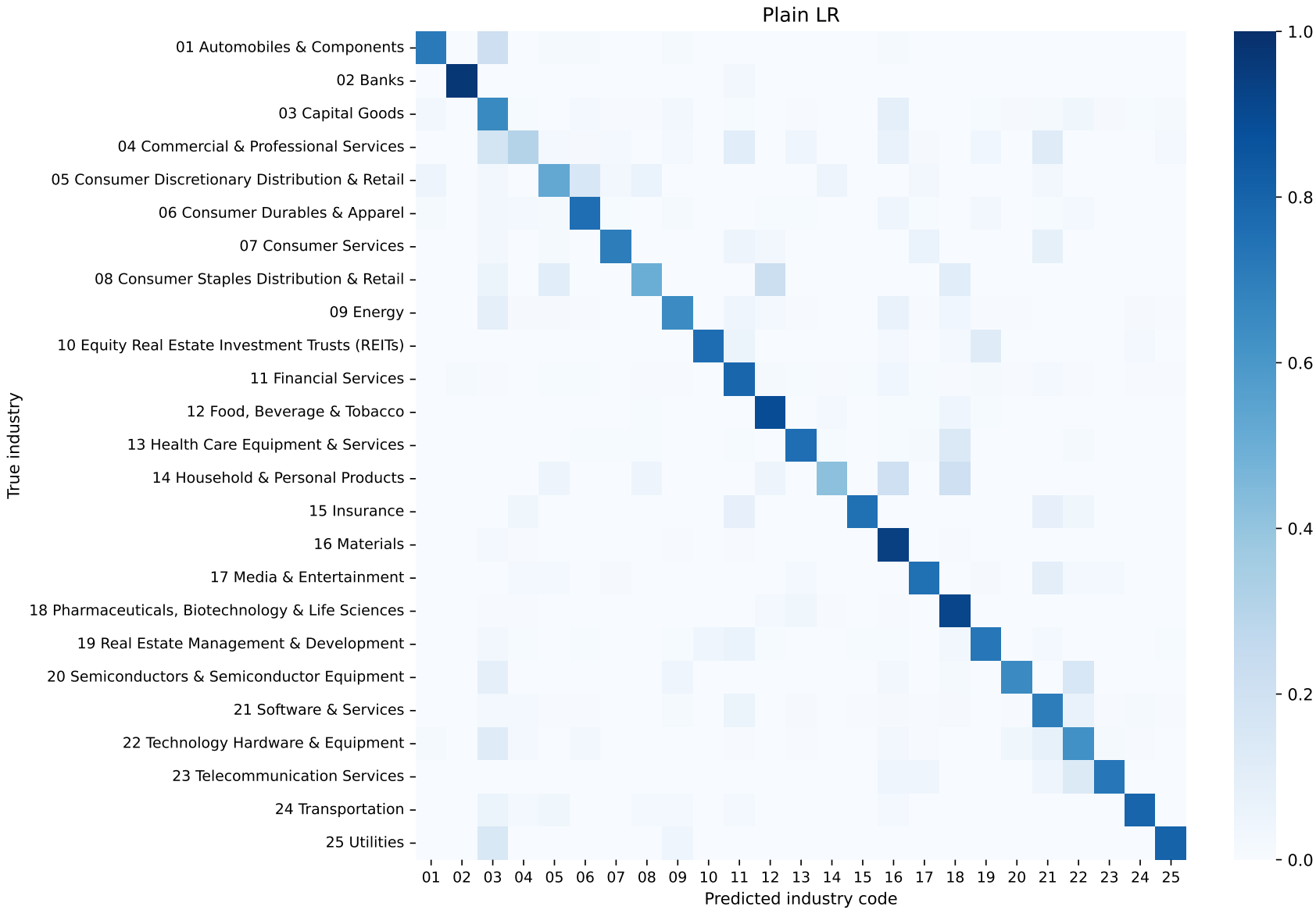
If s_k is the true validation support, the required weighted F1 is $\sum_k (s_k/N) F_{1,k}$. Macro F1 is $K^{-1} \sum_k F_{1,k}$. Accuracy is $N^{-1} \sum_i \mathbf{1}(\hat{y}_i = y_i)$. These are different quantities; "F-measure precision" should be unpacked into precision, recall, and F1.

Required classifier	Accuracy	Weighted F1	Macro F1
Plain LR	0.781457	0.777788	0.717515
Plain k-NN	0.777042	0.771401	0.710471

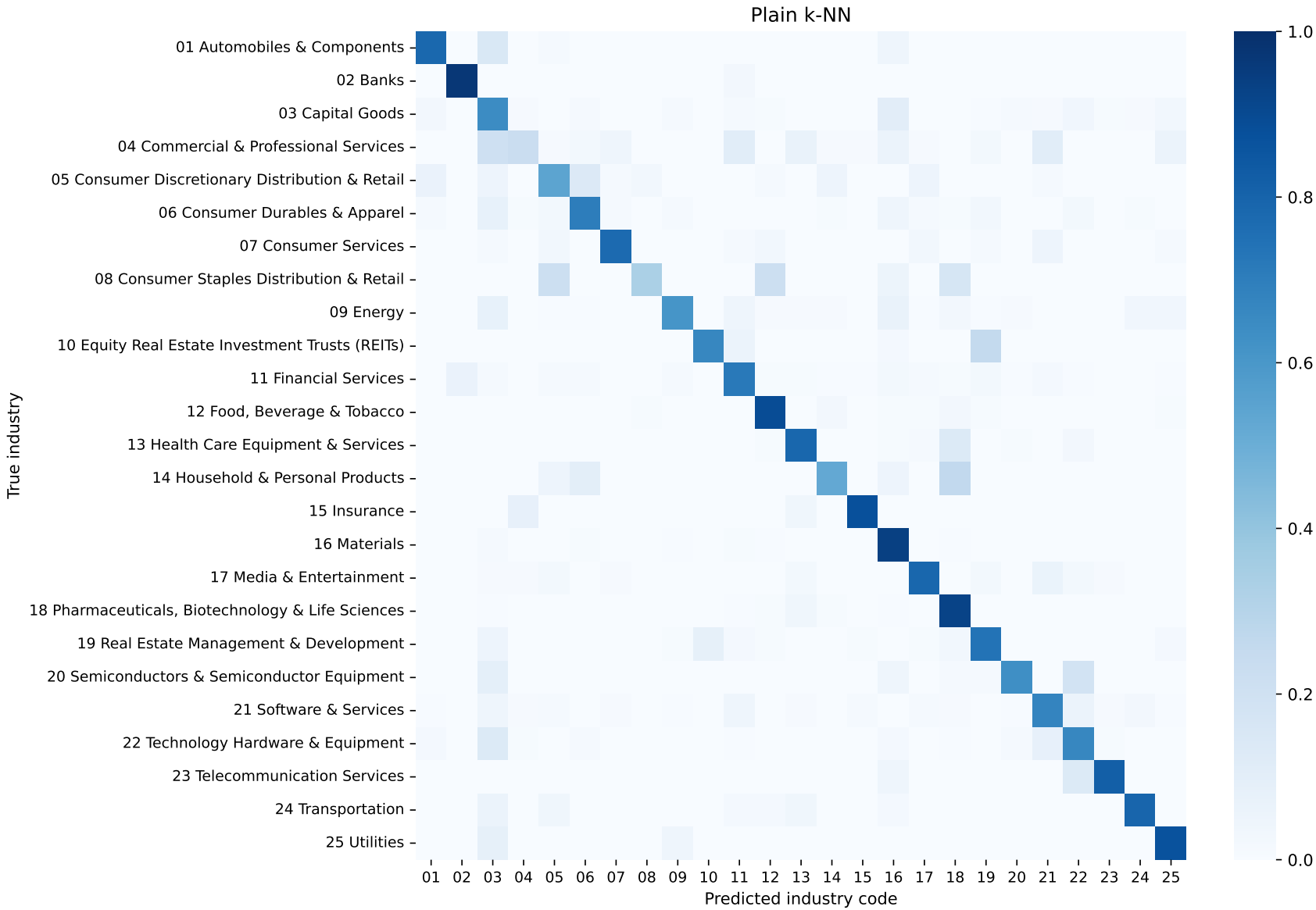
LR is the stronger required standalone classifier here. Macro F1 is lower than weighted F1 for both, showing the role of smaller industries. A class-frequency/error-rate correlation is descriptive: it cannot establish that increasing a particular industry's training count would causally yield the same improvement.

The confusion plots put true classes on rows and predicted classes on columns. Each row is divided by its own support, so a diagonal cell equals recall. The top-error tables instead use raw counts and exclude the diagonal. They describe directed mistakes, not symmetric similarity between industry names.

`matrix.div(matrix.sum(1), axis=0)` creates a normalized table for the plot; it does not overwrite the raw-count matrix. After plotting, the raw diagonal is set to zero. `stack()` then turns each row/column pair into one Series entry, and `nlargest(5)` selects the largest directed error counts. The size correlation similarly relies on pandas aligning industry-name indices before correlating fitting counts with validation `1 − recall`.



Logistic regression: row-normalized validation confusion. Column codes match the row labels.



k-NN: row-normalized validation confusion. Column codes match the row labels.

Model	True industry	Predicted industry	Count
Plain LR	Capital Goods	Materials	28
Plain LR	Technology Hardware & Equipment	Capital Goods	21
Plain LR	Materials	Capital Goods	18
Plain LR	Health Care Equipment & Services	Pharmaceuticals, Biotechnology & Life Sciences	15
Plain LR	Commercial & Professional Services	Capital Goods	15
Plain k-NN	Capital Goods	Materials	33
Plain k-NN	Technology Hardware & Equipment	Capital Goods	24
Plain k-NN	Commercial & Professional Services	Capital Goods	17
Plain k-NN	Materials	Capital Goods	16
Plain k-NN	Technology Hardware & Equipment	Software & Services	15

Both classifiers are wrong on **520** records; **173** errors are unique to LR and **187** to k-NN.

An ensemble can benefit from model-specific errors, but extensive shared errors limit the gain. Having the same total number of errors would not mean that the two models fail on the same companies; the cross-table tests that directly. All classwise results are included later for numerical reference.

Numbered code references

- **[04.01]** Collect comparable aggregate classifier metrics.
- **[04.02]** Use one analysis loop for the two required classifiers.
- **[04.03]** Choose the largest class probability in each row.
- **[04.04]** Record accuracy, support-weighted F1, and equally weighted macro F1.
- **[04.05]** Build per-industry precision, recall, F1, and support results.
- **[04.06]** Display the per-industry report.
- **[04.07]** Compare fitting class size with validation error rate descriptively.
- **[04.08]** Count every true-versus-predicted industry combination.
- **[04.09]** Choose a readable confusion-matrix figure size.
- **[04.10]** Divide each row by its support before plotting.
- **[04.11]** Label the predicted-industry axis.
- **[04.12]** Label the true-industry axis.
- **[04.13]** Identify which classifier produced the confusion matrix.
- **[04.14]** Use smaller horizontal tick labels for the 25 industry names.
- **[04.15]** Use smaller vertical tick labels for the same class order.
- **[04.16]** Keep the matrix labels inside the figure layout.

- **[04.17]** Show the row-normalized confusion plot.
- **[04.18]** Exclude correct predictions from the subsequent error-pair ranking.
- **[04.19]** Display the five most frequent directed industry confusions.
- **[04.20]** Display both classifiers' aggregate metrics together.
- **[04.21]** Count shared and model-specific mistakes on the same companies.

Code block 05: Financial distributions and fitting references

The financial reference data contain **25,680 unique fitting IDs**, with one observation per company and ratio when available. The two text records for one repeated ID remain legitimate distinct text examples, while that ID contributes once to financial profiles. This distinction avoids an accidental extra company in an industry distribution.

The dictionary comprehension is shorthand for two ordinary loops: for each `(industry, group)` produced by `groupby`, visit each ratio column, discard missing observations, convert that Series to an array, and sort it. Keys are two-item tuples `(industry, column)`. Thus 25 industries and three ratios produce 75 reference arrays, each accessible by its industry-and-ratio key.

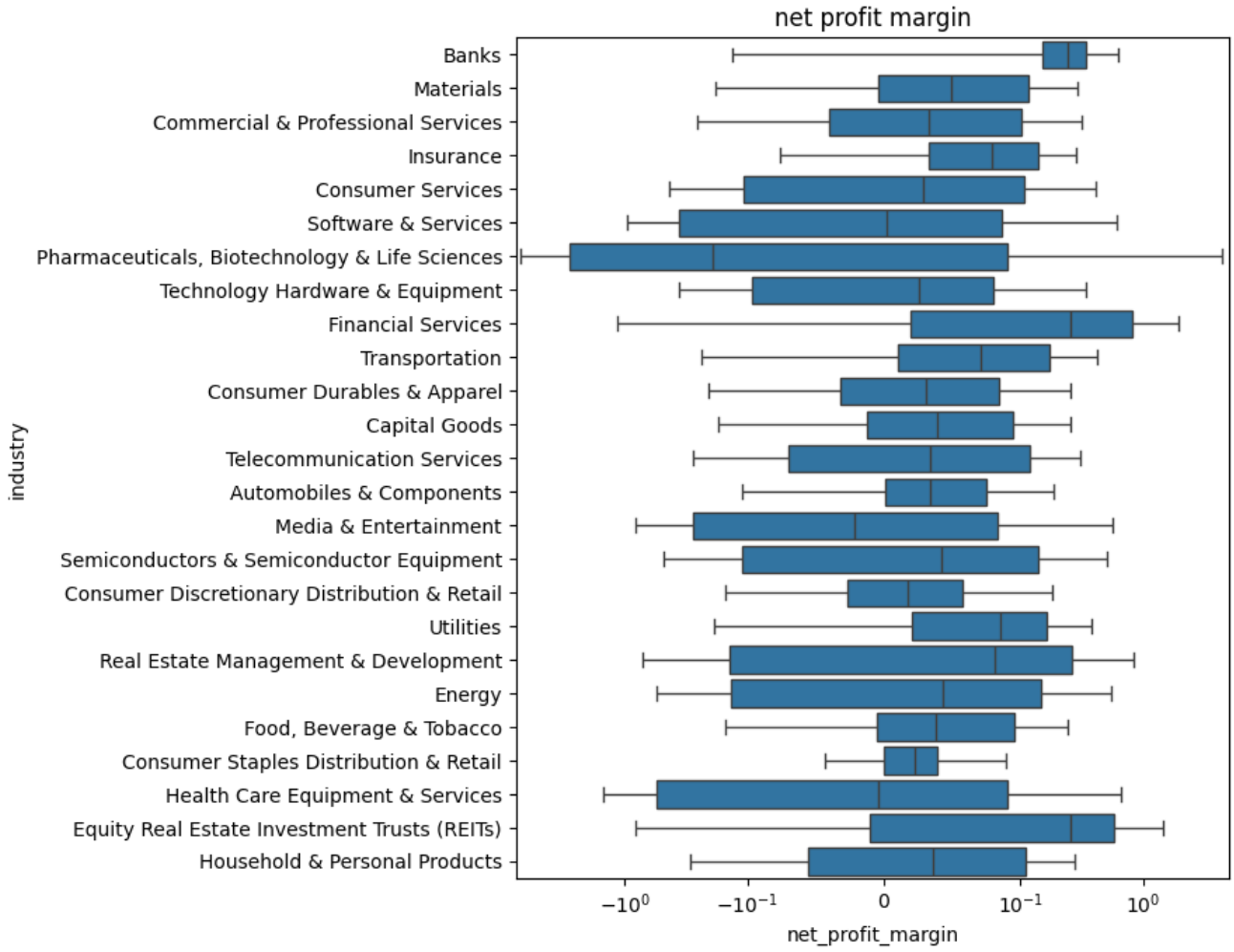


Figure 2: Net profit margin by fitting industry, retaining finite negative values.

A boxplot displays the median, quartiles and a conventional whisker summary. `showfliers=False` hides outlier dots for readability; it does not remove observations from the profile data. Symmetric

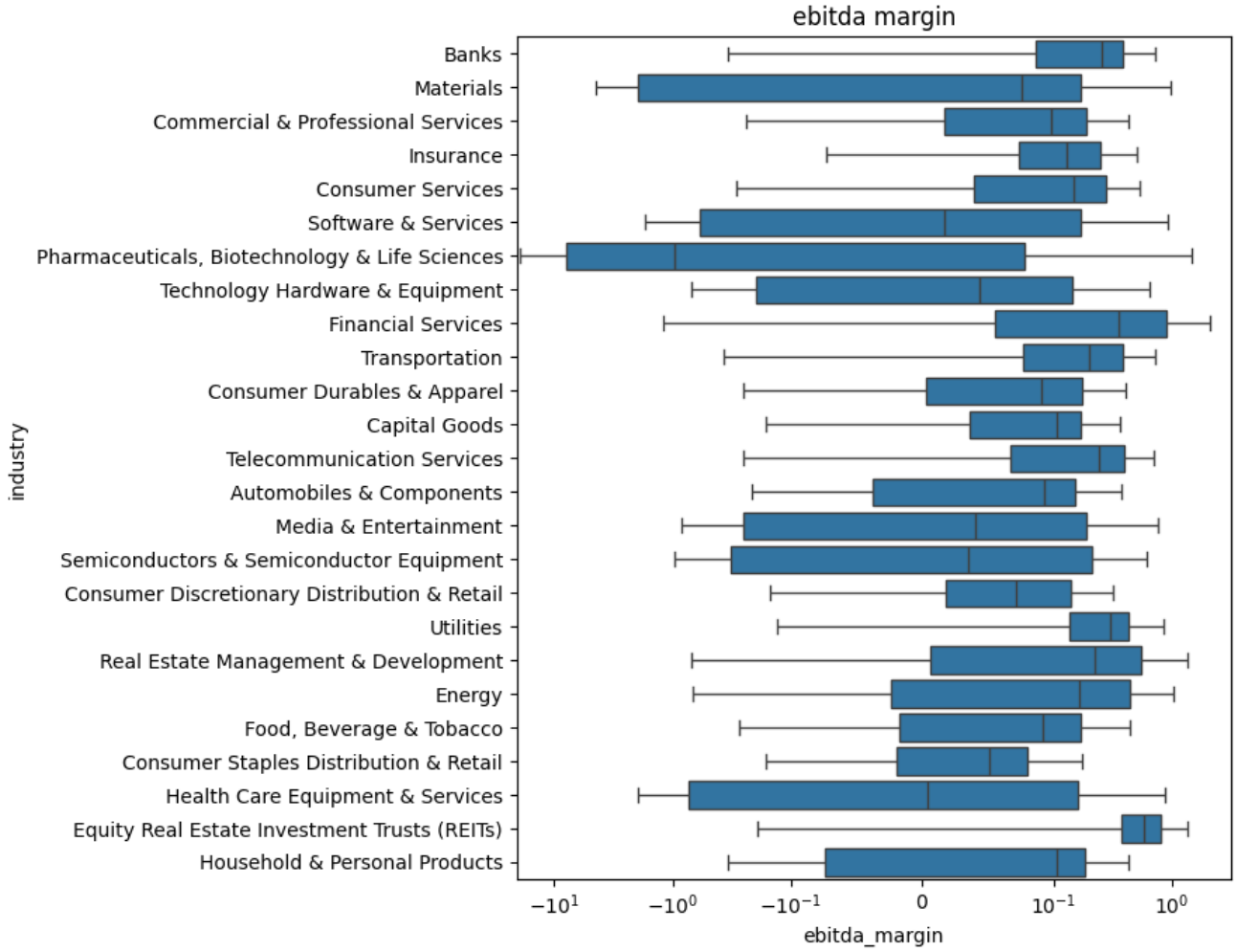


Figure 3: EBITDA margin by fitting industry; coverage is limited.

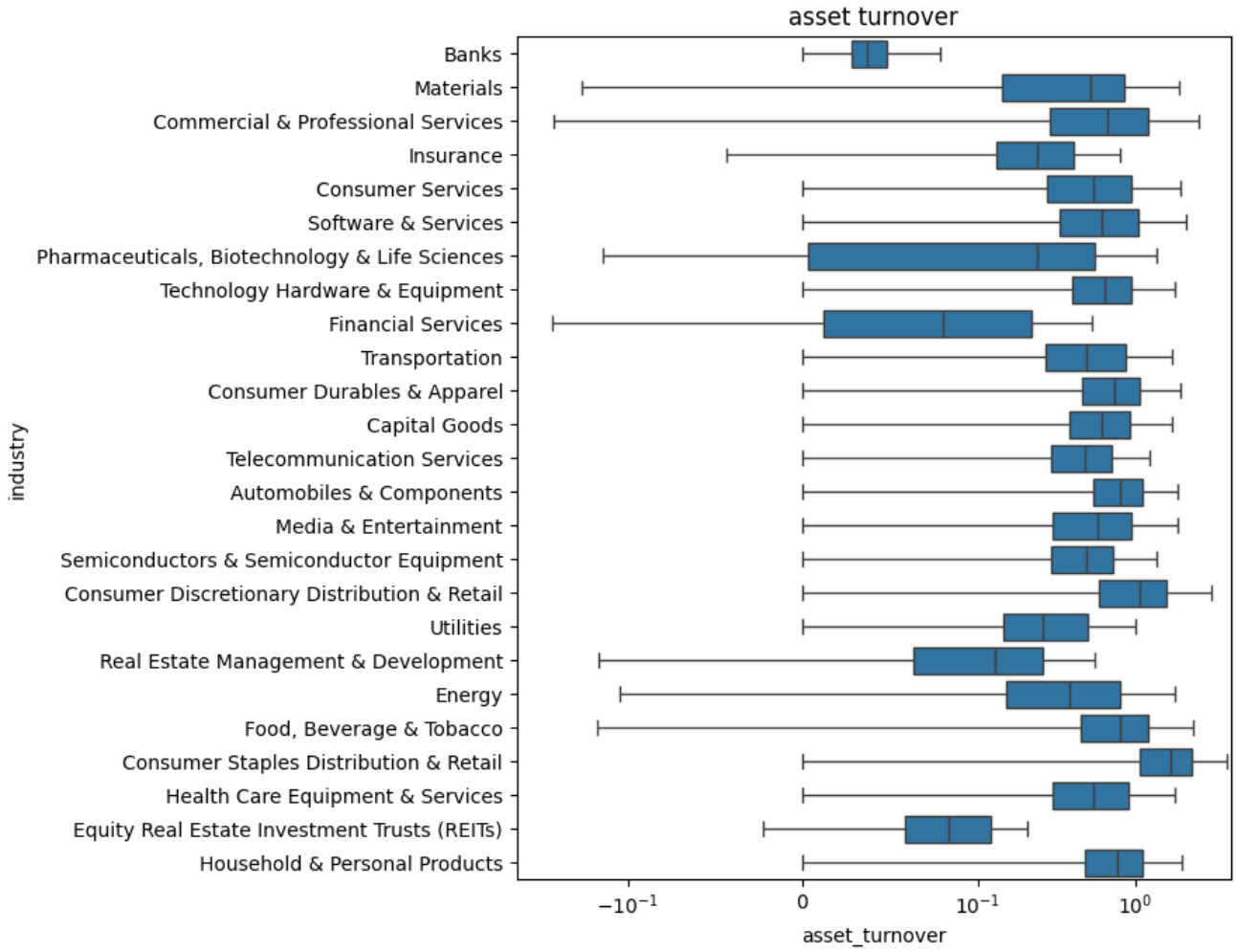


Figure 4: Asset turnover by fitting industry; the plotted axis is symmetric log.

logarithmic scaling permits negative values and spreads large magnitudes without requiring all values to be positive. It changes plotting coordinates, not stored financial values.

The ratios overlap substantially across industries. Net margins include losses, while typical asset turnover differs between financial/property industries and retail. EBITDA margin has particularly poor coverage. These plots motivate weak supporting evidence, rather than a claim that finances alone identify an industry. Across the 75 industry-ratio samples, usable reference counts range from **66 to 6193**.

Numbered code references

- **[05.01]** Use each fitting company ID once in the financial reference data.
- **[05.02]** Display usable financial counts and industry medians.
- **[05.03]** Store sorted observed values for every fitting industry and ratio.
- **[05.04]** Draw one financial-distribution plot per ratio.
- **[05.05]** Choose enough height for the industry labels.
- **[05.06]** Show boxplot summaries while hiding only the outlier dots.
- **[05.07]** Keep losses and extreme positive values visible with symmetric logarithmic scaling.
- **[05.08]** Use a readable financial-ratio title.
- **[05.09]** Adjust the plot layout for the long industry labels.
- **[05.10]** Show this financial-distribution plot.

Code block 06: The score and statistical confidence inputs

The official score

References [06.01]–[06.09] implement the current course definition. Arrays must have equal lengths, and confidence must be in [0,1]. NaN and infinite values fail the bound check. Weighted F1 uses true class support. Correctness Brier uses one confidence per selected label. There is no filtering or confidence-weighted confusion table in this definition.

For true correctness chance p, expected Brier loss obeys

$$\mathbb{E}[(c - Z)^2] = (c - p)^2 + p(1 - p).$$

Its minimum is at c=p. This gives a reason to estimate honest probabilities, rather than assign low confidence solely after seeing which labels are wrong. For fixed predictions, $S \leq F_{1,\text{weighted}}$ because Brier is nonnegative. With the current labels the maximum possible score, even with oracle correctness confidence, is **0.780881**. An oracle is an upper bound, not a practical calibration method.

Vote and ambiguity

The retained class vote is $v_k = 0.25p_k^{LR} + 0.75p_k^{NN}$. The selected industry is its largest entry. Both matrices use exactly the same encoded class order, so this sum combines the same industries. Their weights sum to one, preserving a class-probability vector.

The largest vote q measures concentration. The difference between its largest and second-largest entries measures ambiguity between leading alternatives. Agreement records whether the two base classifiers choose the same industry. Nearest distance adds proximity information not fully expressed by the vote.

Empirical financial compatibility

For an observed ratio x and its predicted industry's sorted fitting sample, define L as the number of reference values less than x and R as the number less than or equal to x. With sample size n,

$$u = \frac{L + R}{2n}, \qquad t = 2\min(u, 1 - u).$$

This is an empirical mid-rank followed by central-rank compatibility. Values outside the observed range have t=0, a middle rank gives t=1, and ties are split symmetrically. With reference [1,2,2,4] and x=2, L=1 and R=3, so u=0.5 and t=1. This small example illustrates ties; the worked company cases later use actual financial observations and sample sizes.

Only finite ratios for the predicted industry contribute. Available t values are averaged, and availability is usable-count divided by three. With no evidence, the average placeholder is 0.5 and availability is zero. Empty reference samples are skipped. The actual data have at least 66 usable observations in every industry-ratio sample, but a general empty-sample guard avoids division by zero.

The mask in [06.21] is an N-element Boolean array. It selects rows whose predicted name matches the current reference industry and whose current ratio is finite. & combines those conditions elementwise. Updates to `total[mask]` and `available[mask]` therefore affect exactly those input positions. Each row can collect at most three contributions. The function's local financial table contains only the selected ratio columns; the original input and its row order are retained.

`np.divide` uses an output initially filled with 0.5 and divides only where the observed count is positive. This avoids dividing by zero even temporarily. Simply evaluating `total/available` before choosing missing-case values would calculate invalid divisions on the all-missing rows.

The rank measure is bounded, robust to the magnitude of extreme tails, and unchanged by a strictly increasing transformation applied consistently to the reference and observation. It is not a density, a likelihood, a posterior industry probability, or a formally calibrated p-value. A location between two modes can have a middle rank despite low density. The assignment permits other statistical measures, so that narrower interpretation is sufficient and more accurate than labeling the feature a p-value.

Six features in a fixed order

Position	Feature	Meaning
1	Largest-vote log-odds	log(q/(1−q)), after clipping only to avoid infinite logarithms
2	Top-two vote margin	Largest vote minus second-largest vote
3	LR/k-NN agreement	One when both required classifiers choose the same label
4	Log-one-plus nearest distance	log(1+d), using the nearest PCA-space fitting neighbor
5	Financial compatibility	Mean observed predicted-industry central-rank score; 0.5 with no evidence
6	Financial availability	Usable financial ratios divided by three

Log-odds require q to lie strictly between zero and one; clipping at 10^{-6} and $1-10^{-6}$ prevents infinite logarithms. It does not clip financial observations. `log1p` is well-defined at nearest distance zero. The feature function returns predicted codes and an $N \times 6$ matrix, reads no true industry label, and handles absent financial columns explicitly.

Sorting along axis 1 sorts the 25 votes within each row; indices −1 and −2 select its largest and second-largest values. `column_stack` places the six one-dimensional feature arrays beside each other, converting Boolean agreement to numerical zero or one. None of these operations sorts companies or changes the row alignment.

Numbered code references

- **[06.01]** Define the current course confidence-score interface.
- **[06.02]** Convert true and predicted labels to arrays for elementwise comparison.
- **[06.03]** Convert the confidence vector to floating-point values.
- **[06.04]** Check that each target has a prediction and a confidence.
- **[06.05]** Reject values outside the required finite probability interval.
- **[06.06]** Compute F1 weighted by the true class supports, without confidence weights.
- **[06.07]** Represent prediction correctness as one and error as zero.
- **[06.08]** Measure mean squared correctness-probability error.
- **[06.09]** Apply exactly weighted F1 times one minus Brier.
- **[06.10]** Construct confidence evidence without reading any true input industry.
- **[06.11]** Unpack both class-probability matrices and nearest-neighbor distances.
- **[06.12]** Mix the aligned class probabilities with the selected LR weight.
- **[06.13]** Choose the class and sort votes to recover the largest two values.
- **[06.14]** Translate predicted class codes to the financial-profile industry names.
- **[06.15]** Represent absent financial columns as missing values.
- **[06.16]** Start each row’s compatibility total and evidence count at zero.
- **[06.17]** Visit the fitting-only industry-ratio reference samples.
- **[06.18]** Check whether a reference contains any observed fitting values.
- **[06.19]** Skip an empty reference rather than divide by zero.
- **[06.20]** Read this ratio in exactly the input-row order.
- **[06.21]** Use finite values for the predicted industry only.
- **[06.22]** Count reference observations strictly below each value.
- **[06.23]** Count reference observations below or equal to each value.
- **[06.24]** Calculate empirical mid-ranks, including ties.
- **[06.25]** Add bounded central-rank compatibility for this observed ratio.
- **[06.26]** Count this usable financial contribution.
- **[06.27]** Average available evidence and use 0.5 only when none is present.
- **[06.28]** Clip the largest vote solely to keep its log-odds finite.
- **[06.29]** Combine log-odds, margin, agreement, distance, financial rank, and availability.
- **[06.30]** Return industry codes and the six confidence-feature columns.

Code block 07: Learning correctness confidence

The calibration target is $z_i = \mathbf{1}(\hat{y}_i = y_i)$. Of 2,854 calibration records, **2283** are correct under the fixed selected vote. These known labels teach correctness; at inference they are unavailable and unused.

Each feature h_j is standardized using calibration mean μ_j and scale s_j. The fitted binary logistic model then computes

$$\tilde{h}_j = (h_j - \mu_j)/s_j, \quad \eta = b + \sum_{j=1}^{6} \beta_j \tilde{h}_j, \quad c = \sigma(\eta) = \frac{1}{1 + \exp(-\eta)}.$$

`StandardScaler` uses the population-variance convention for its fitted scales and handles constant features. The pipeline keeps scaling and the classifier together, so inference reuses the same transformation. Logistic fitting minimizes regularized binary log loss; it does not directly minimize the reported Brier. Both are proper probability losses in their ideal population formulation.

Feature	Calibration mean	Scale	Coefficient
Largest-vote log-odds	2.3841318	2.5353740	1.5288688
Top-two vote margin	0.6522222	0.3159755	0.2635486
LR/k-NN agreement	0.8710582	0.3351356	0.0422727
Log-one-plus nearest distance	0.3078832	0.1450038	-0.1919633
Financial compatibility	0.4825750	0.2097535	0.0889965
Financial availability	0.6702873	0.2967985	0.1253243

The intercept is **2.196062555** on standardized features. Its sigmoid is the probability at the vector of feature means, not the overall mean correctness: averaging a nonlinear sigmoid is not the same as applying it to averages. Coefficients describe conditional associations, not causal effects. Their units are calibration standard deviations rather than raw financial-ratio units. Correlated features can alter coefficient sizes. The correct numerical prediction sums all standardized contributions and intercept; inspecting a single sign cannot substitute for that calculation.

The text-only ablation uses the same records, selected labels, model family, and first four features. Only financial compatibility and availability are removed. This is a controlled comparison of the incremental financial inputs, rather than a comparison confounded by different company populations or different ensemble labels.

The binary target has both False and True outcomes. Scikit-learn orders those classes as False, True, so probability column `[:, 1]` estimates the True class: a correct selected label. This column index belongs to the binary correctness model, not to industry code 1 in the 25-class classifiers.

Numbered code references

- **[07.01]** Use the development-selected 25 percent LR and 75 percent k-NN vote.
- **[07.02]** Build labels and confidence inputs on the entire held-out calibration pool.
- **[07.03]** Scale confidence features before regularized correctness logistic regression.
- **[07.04]** Learn correctness, rather than industry labels, on calibration records only.
- **[07.05]** Create the same model for the text-only financial-evidence ablation.
- **[07.06]** Fit that ablation using the same records and first four features.

Code block 08: Evaluation, calibration, and review

The table at the start gives the exact local metric comparison. The selected financial model has Brier **0.13353370**. Its score improvement over the text-only ablation is **0.00082295**, or **0.0823 percentage points**. This supports a small local contribution; it does not establish a large or universal financial benefit. Raw confidence, a calibrated text model, and a calibrated financial model share the same ensemble labels.

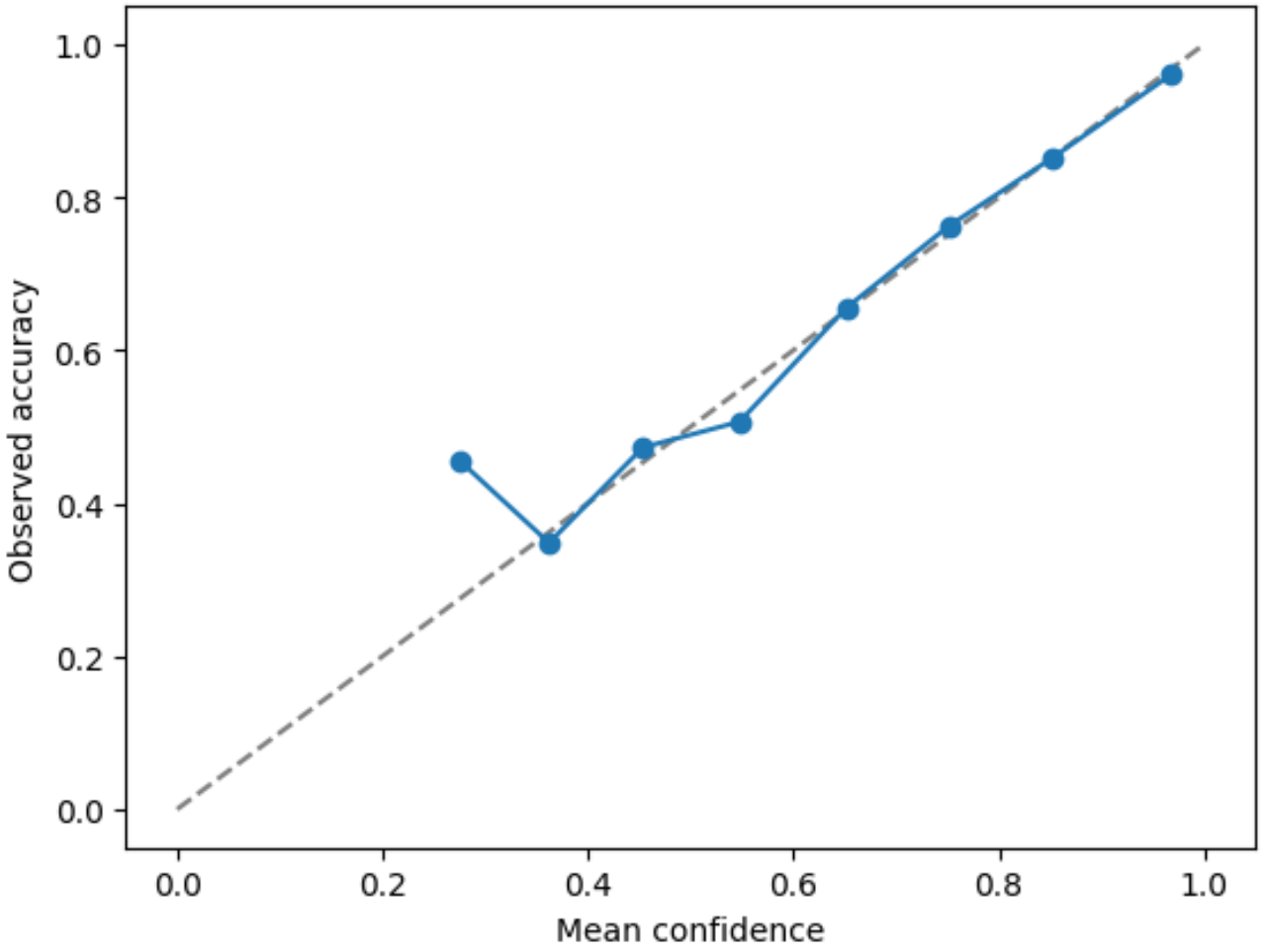


Figure 5: Confidence versus observed validation accuracy.

Occupied bin	Count	Mean confidence	Accuracy
0.2–0.3	11	0.275965	0.454545
0.3–0.4	89	0.362116	0.348315
0.4–0.5	222	0.453534	0.472973
0.5–0.6	306	0.547672	0.506536
0.6–0.7	325	0.651478	0.655385
0.7–0.8	401	0.752062	0.763092
0.8–0.9	619	0.852042	0.851373
0.9–1.0	1198	0.968253	0.960768

The largest bins are informative, while the lowest occupied bin has only 11 records. The 0.8–0.9 bin's mean confidence is close to its observed accuracy; the top bin is slightly overconfident. Binning is a coarse display, and a near-diagonal line is not proof of perfect calibration or transfer to the private test. Continuous Brier provides a complementary summary. Overall mean confidence is **0.789727**, compared with accuracy **0.786187**. These are respectively the count-weighted means of the bin confidences and bin accuracies. Aggregate agreement alone can hide offsetting discrepancies between bins.

For these tables, `size` counts all records in a group, and the mean of the Boolean `Correct` column is its accuracy. `observed=True` omits unoccupied categorical bins from the display; it does not remove any observations from the official score. The cut boundaries include confidence zero, and the endpoint at one belongs to the highest bin.

The fixed confidence-below-0.8 policy flags **1,354 of 3,171 records (42.70%)** and captures **539 of 678 errors (79.50%)**. Flagged accuracy is **60.19%** and retained accuracy **92.35%**. Review workload is therefore substantial. All records remain in the course score. Selective retained accuracy is not the official score and cannot be substituted for a bonus threshold.

For a real review system, an explicit cost model and development-only policy selection would be appropriate. This notebook demonstrates the requested relationship between confidence, accuracy, and review; it does not present 0.8 as an instructor requirement or a globally optimal cutoff.

Numbered code references

- **[08.01]** Build the selected ensemble's validation labels and confidence features.
- **[08.02]** Read the fitted probability of the correct-prediction class.
- **[08.03]** Compare plain classifiers, raw voting, and both calibrated confidence methods.
- **[08.04]** Compute weighted F1, correctness Brier, and the official score for each method.
- **[08.05]** Create a compact table of the actual validation results.
- **[08.06]** Calculate the paired-split score difference from plain LR.
- **[08.07]** Display the baseline and ensemble results together.
- **[08.08]** Pair each validation confidence with its observed correctness.
- **[08.09]** Summarize occupied fixed-width confidence bins with counts and accuracy.
- **[08.10]** Display the reliability-bin evidence, including sample sizes.
- **[08.11]** Draw the ideal confidence-equals-accuracy reference.
- **[08.12]** Plot the model's observed reliability points.
- **[08.13]** Label the average-confidence axis.
- **[08.14]** Label the observed-correctness axis.
- **[08.15]** Show the reliability plot.

- **[08.16]** Flag confidence below 0.8 under a fixed illustrative review policy.
- **[08.17]** Compare flagged and retained groups without removing records from scoring.
- **[08.18]** Report the review workload and how many errors it captures.

Code block 09: Inference and limits of representation novelty

`predict_industry(companies)` first parses embeddings, reuses the fitted PCA and base classifiers, computes the selected vote and financial features, and uses the fitted correctness pipeline. It returns industry names and confidence as two vectors in input-row order. It never reads the true `industry` field. The example deliberately drops that field.

The required input is `business_description_embedding` in the supplied string format. Financial ratio columns are optional. If separate financial records are provided, aggregate and join them by ID before prediction, as in block 02. The returned names must be compared to true industry names, or encoded with the fitted encoder before comparison to integer `y_val`.

455 validation records (14.35%) share an exact stored embedding string with fitting records. Accuracy is **99.34%** on those and **75.15%** on unseen stored strings. This is an audit of exact stored strings, not a guarantee that all other descriptions are semantically distinct. It shows why an ID-disjoint score can be easier than performance on entirely new business representations.

The prediction function was checked for matching one-dimensional output shapes, finite bounded confidence, row-order preservation, independence from an arbitrary true-label column, and equivalent treatment of absent, NaN, and infinite financial evidence. These checks are distinct from performance estimation and do not use synthetic scores as assignment results.

Numbered code references

- **[09.01]** Expose the required prediction-and-confidence tuple interface.
- **[09.02]** Reuse the parser, fitted base models, vote, and financial evidence.
- **[09.03]** Return industry names and correctness probabilities in input-row order.
- **[09.04]** Demonstrate inference after removing the true industry labels.
- **[09.05]** Display three predicted industry/confidence pairs.
- **[09.06]** Identify exact stored embedding strings also present among fitting records.
- **[09.07]** Compare accuracy on seen and unseen stored representations.
- **[09.08]** Report elapsed notebook time after imports.

Worked predictions and calculations

These cases use the actual fitted validation outputs. Positions refer to the local validation array, not company IDs. True industries are shown only to explain evaluation; the prediction function did not receive them. Rounded arithmetic can differ in the last digit from full-precision results.

Case A: a correct prediction

Validation position **2** has true industry **Consumer Services**. The ensemble predicts **Consumer Services**.

Industry	LR probability	k-NN probability	25/75 vote
Consumer Services	0.88306143	0.87150562	0.87439457
Consumer Discretionary Distribution & Retail	0.00350119	0.12849438	0.09724608
Financial Services	0.03547311	0.00000000	0.00886828

The leading vote is $0.25 \times 0.88306143 + 0.75 \times 0.87150562 = 0.87439457$. The top-two margin is **0.77714849**, and LR/k-NN agreement is **1**. The next financial calculations use this predicted industry, not the true one [06.12]–[06.29].

Ratio	Value	n	L / R	u	Compatibility
net profit margin	0.01862862	469	211 / 211	0.4498934	0.8997868
ebitda margin	0.09684027	260	92 / 92	0.3538462	0.7076923
asset turnover	0.67085282	477	284 / 284	0.5953878	0.8092243

Their mean compatibility is **0.80556780**, and availability is **1.0000**. The six-feature standardization and coefficient contributions are:

Feature	Raw h	Standardized h	Coefficient × standardized h
Largest-vote log-odds	1.9403863	-0.1750217	-0.2675853
Top-two vote margin	0.7771485	0.3953671	0.1041984
LR/k-NN agreement	1.0000000	0.3847453	0.0162642
Log-one-plus nearest distance	0.5071198	1.3740098	-0.2637594
Financial compatibility	0.8055678	1.5398687	0.1370429
Financial availability	1.0000000	1.1108973	0.1392224

The intercept **2.196062555** plus those contributions gives $\eta = 2.061445795$. Applying the sigmoid gives

$$c = \frac{1}{1 + \exp(-2.061445795)} = 0.887099054.$$

Correctness is **1**, so this record contributes $(0.887099054 - 1)^2 = 0.012746624$ to the Brier average. It is retained by the 0.8 policy.

Case B: an incorrect prediction

Validation position **0** has true industry **Technology Hardware & Equipment**. The ensemble predicts **Capital Goods**.

Industry	LR probability	k-NN probability	25/75 vote
Capital Goods	0.32255329	0.72524570	0.62457260
Technology Hardware & Equipment	0.55181219	0.13901347	0.24221315
Consumer Durables & Apparel	0.05461638	0.13574082	0.11545971

The leading vote is $0.25 \times 0.32255329 + 0.75 \times 0.72524570 = 0.62457260$. The top-two margin is **0.38235945**, and LR/k-NN agreement is **0**. The next financial calculations use this predicted industry, not the true one [06.12]–[06.29].

Ratio	Value	n	L / R	u	Compatibility
net profit margin	0.11988204	2276	1876 / 1876	0.8242531	0.3514938
ebitda margin	0.16179975	727	521 / 521	0.7166437	0.5667125
asset turnover	1.33823865	2305	2076 / 2076	0.9006508	0.1986985

Their mean compatibility is **0.37230162**, and availability is **1.0000**. The six-feature standardization and coefficient contributions are:

Feature	Raw h	Standardized h	Coefficient × standardized h
Largest-vote log-odds	0.5090025	-0.7395869	-1.1307313
Top-two vote margin	0.3823594	-0.8540621	-0.2250868
LR/k-NN agreement	0.0000000	-2.5991219	-0.1098719
Log-one-plus nearest distance	0.3108299	0.0203219	-0.0039011
Financial compatibility	0.3723016	-0.5257286	-0.0467880
Financial availability	1.0000000	1.1108973	0.1392224

The intercept **2.196062555** plus those contributions gives $\eta = 0.818905878$. Applying the sigmoid gives

$$c = \frac{1}{1 + \exp(-0.818905878)} = 0.694004039.$$

Correctness is **0**, so this record contributes $(0.694004039 - 0)^2 = 0.481641606$ to the Brier average. It is flagged by the 0.8 policy.

This is a concrete remaining error, not removed from the reported F1 or official score. The LR top class is correct in this case, but the larger k-NN vote changes the ensemble label. Ensembling can introduce some mistakes as well as correct others.

A real per-industry F1 calculation

For LR and Banks: TP=103, FP=3, FN=3, support=106. Thus precision is $103/(103 + 3) = 0.97169811$, recall is $103/(103+3) = 0.97169811$, and F1 is $2(103)/(2(103)+3+3) = 0.97169811$. Its weighted-F1 contribution is $(106/3171) \times 0.97169811$. Summing the equivalent terms over all 25 industries gives the reported weighted F1.

Calculating the official score

The selected model has $F_1 = 0.7808814719$ and $B = 0.1335337022$. Therefore

$$S = 0.7808814719(1 - 0.1335337022) = 0.6766074779.$$

Subtracting the same-split plain-LR score gives 0.0054443655. Multiplying the difference by 100 converts it to 0.5444 percentage points. F1, the official score, a relative percentage change, and percentage-point change are not interchangeable.

Oral-exam questions and answers

The problem and the supplied data

1. What exactly does this project predict?

One industry from the 25 labels represented in the fitting data, together with an estimate of whether that selected label is correct. Industry prediction is multiclass classification. Confidence fitting is a separate binary problem: correct versus incorrect. The two output arrays answer different questions.

2. Why does the notebook report fewer than 40,000 companies?

The assignment describes the broader course dataset. The supplied main pickle has 31,706 records and 31,705 unique company IDs. Those are the actual observations available here. I report that scope rather than invent extra companies, silently substitute financial rows, or treat the assignment's headline count as an observed count.

3. What is an embedding?

A numerical representation whose coordinates jointly describe a text's meaning. Here each business description is already represented by 768 coordinates. Semantically related descriptions can have similar vectors, but no single coordinate is guaranteed to have a human-readable financial interpretation. Embeddings are features, not industry labels or correctness probabilities.

4. Which language model generated the embeddings?

The supplied embedding example uses `sentence-transformers/all-mpnet-base-v2`. Its output dimension is consistent with the data. However, the pickle alone does not certify the generation model, version, or preprocessing, and other encoders can also produce 768 coordinates. This project consumes the supplied vectors; it does not claim to have regenerated them.

5. What does "pretrained" mean here?

The text encoder learned a representation before this classification task. We reuse its output and fit comparatively small task-specific models. The notebook does not train a large language model, call a paid text API, or require raw business descriptions or encoder weights at prediction time.

6. Why not use company ID as a predictor?

An identifier distinguishes records but has no meaningful numerical business geometry. Treating ID differences as financial distances could encourage memorization and fail on new IDs. IDs are used to join records, keep companies together during splitting, and deduplicate financial references, rather than as classifier features.

7. What are the three financial ratios?

Net profit margin relates net profit to revenue, EBITDA margin relates earnings before interest, tax, depreciation and amortization to revenue, and asset turnover relates revenue to assets. The notebook uses the supplied values; the raw accounting numerators and denominators are not available to recalculate or independently audit them.

8. Why does the project not use country code?

It is not needed for the required embedding comparison or the chosen financial-rank evidence. Geographic coding can also introduce patterns that may not transfer. Adding a column merely because it exists would make the solution larger without demonstrating a reliable benefit. The feature contract lists exactly what prediction uses.

9. What is the difference between a DataFrame, a Series, and an array?

A DataFrame is a labeled two-dimensional table. Selecting one column normally yields a one-dimensional Series with its index. NumPy arrays hold numerical values by position. This matters because pandas may align operations by index labels, while the classifier arrays must preserve exactly the same row order as the target arrays.

10. Why use `iloc` after splitting?

`GroupShuffleSplit` returns row positions. `iloc` selects those positions even when the DataFrame's index labels are not consecutive. Using `loc` would interpret the numbers as labels and could select different rows or fail. Predictions and financial masks are subsequently handled in consistent positional order.

11. Why parse embeddings with `ast.literal_eval`?

The supplied column contains strings representing lists, rather than numerical arrays. Literal parsing converts those lists to numbers without evaluating arbitrary Python expressions. The array constructor then assembles one row per company. The provided vectors were checked for a common length and finite coordinates.

12. Why store embeddings as float32?

Each coordinate uses four bytes rather than eight, reducing memory for the large dense matrix. For 31,706 by 768 entries, the float32 matrix occupies about 93 MiB. Parsed strings and temporary Python objects also consume memory, so matrix size alone is not a measurement of process peak memory.

Cleaning, splitting, and leakage

13. Why distinguish NaN from infinity in the initial table?

NaN means an unavailable numerical value; positive or negative infinity is also unsuitable as a finite financial observation. Their raw frequencies describe different data-quality problems. After inspection both become missing for the chosen analysis, while legitimate finite values, including losses, remain available.

14. Why use medians for repeated financial IDs?

The financial file can contain more than one record for a company. Grouping by ID gives one reference row and prevents a join from multiplying company records. The median is less sensitive to extremes than the mean. Without dates or accounting-period metadata, it is a transparent aggregation rule rather than a claim to identify the latest report.

15. Does aggregation across repeated financial records leak industry labels?

It does not use industry labels or global fitted imputation statistics. It summarizes the same company's supplied records before the company-group split. Learned population quantities, including PCA axes, classifier parameters, financial industry reference distributions, and confidence scaling, are fitted only in their designated training subsets.

16. Why is the financial join a left join?

The main company table defines the prediction population. A left join keeps companies that lack a financial record and gives them missing financial fields. An inner join would discard those harder cases and change the population being evaluated. Confidence explicitly records financial availability instead.

17. What does `validate='many_to_one'` check?

Many main records may share an ID, but the aggregated financial table must have at most one row for each ID. The validation argument rejects a join that violates that relationship. It protects row counts and label alignment from accidental multiplication caused by unaggregated financial duplicates.

18. Why not split individual rows at random?

The same company ID can occur more than once. A row split could put its records on both sides, weakening the meaning of unseen-company evaluation. Group splitting keeps those records together. It does not, however, ensure that different IDs have different business descriptions; that limitation is measured separately.

19. Why have fitting, calibration, and validation sets?

Fitting learns PCA, industry classifiers, and financial reference samples. Calibration learns the relationship between observable evidence and correctness. Validation estimates behavior after those choices

are fixed. Learning confidence on the same records used to fit the industry models would generally make correctness look easier than it is on new companies.

20. Where did tuning happen if the final notebook has only three sets?

During development, the calibration pool was split into separate calibration and tuning halves. Candidate confidence models fitted on one half and were compared on the other using the official score. The selected configuration was then fixed and its final confidence model refitted on the whole calibration pool. Validation did not select those settings.

21. Is `GroupShuffleSplit` stratified by industry?

No. It separates company groups rather than guaranteeing exact class proportions. The actual data were checked to include every industry in the fitting and evaluated subsets. Class-support tables and macro F1 reveal imbalances. A different dataset might require a deliberately stratified group split or a revised evaluation design.

22. Why are the percentage sizes approximate?

The fractions refer to groups and are applied in stages. Ten percent is reserved first, then ten percent of the remaining development groups is held out for calibration. This gives about 81%, 9%, and 10% overall. Integer rounding and repeated IDs make the record counts slightly different from exact products of the total.

23. Why keep random state 42?

It makes the split and specified randomized operations repeatable. The number itself has no statistical advantage. Reproducibility does not mean that a single split represents all possible populations, nor does a seed eliminate differences caused by library versions, numerical backends, or decisions made after inspecting results.

24. Can I call this a completely untouched test set?

The same local holdout was examined in previous project versions, so it is a revised local validation result. Individual settings in the current comparison were selected on development data, but repeated project-level inspection limits independence. The professor's separate private test is the external assessment. These distinctions are stated rather than hidden.

25. Does a company-ID split remove all repeated representations?

No. Distinct companies can have identical stored embedding strings. The notebook reports their validation frequency and seen-versus-unseen accuracy. This evaluates new IDs, not exclusively new representations. Such overlap can make overall local accuracy more optimistic about wholly novel descriptions.

PCA and classification

26. What does PCA optimize?

It finds orthogonal directions that retain the most fitting-data variance under a linear projection. Equivalently, the leading components minimize squared reconstruction error among projections of that dimension. It is unsupervised: industry labels do not determine its axes. Maximum variance is not the same objective as maximum classification accuracy.

27. Why must PCA be fitted before transforming validation?

The fitting set determines the mean and component directions. Applying that already fitted mapping to validation imitates how a new company is processed. Fitting PCA on all embeddings would let held-out feature distributions influence preprocessing, even without using their labels, and would change the intended separation.

28. How is explained variance calculated?

Each component has an associated covariance eigenvalue. The fraction for component j is its eigenvalue divided by the sum over all original components. Cumulative retained variance sums those

fractions for the retained components. It describes feature variation, rather than a percentage of correct labels or a guarantee about preserved discriminative information.

29. Why show two components but retain 128?

Two axes produce a readable scatterplot. More axes retain information for distance-based classification. The development comparison favored the retained PCA representation for the chosen ensemble. Logistic regression retains all 768 original coordinates, allowing the two required classifiers to use different representations.

30. Does overlap in the scatterplot prove that industries are inseparable?

No. A two-dimensional projection discards most coordinates and may hide useful boundaries. Conversely, visible clusters do not establish high out-of-sample accuracy. The plot motivates questions about geometry; held-out classification metrics measure predictive performance. The guide includes the actual cumulative variance values to quantify the reduction.

31. Why can PCA axes have their signs reversed?

An eigenvector and its negative describe the same one-dimensional subspace. Reversing both a projected coordinate and its axis orientation changes the appearance but not retained variance or pairwise Euclidean distances in the retained space. Axis sign has no intrinsic economic meaning.

32. Why not standardize all embedding coordinates first?

These supplied vectors are already approximately unit norm, and their coordinate geometry comes from the pretrained encoder. Arbitrarily giving each coordinate unit variance would change that geometry. PCA centers fitting vectors internally. Separate standardization in the notebook applies only to confidence features with intentionally different numerical units.

33. Is logistic regression a regression model or a classifier here?

It is a classifier that models class probabilities through linear scores and a probability link. The industry model uses a multiclass softmax. The correctness model uses a binary sigmoid. Neither fits a straight line directly to arbitrary industry code numbers, and neither treats those codes as ordinal financial quantities.

34. What does multiclass logistic regression learn?

One linear score per class, consisting of coefficients and an intercept. Softmax normalizes exponentials of those scores across classes. Training minimizes regularized classification log loss. The largest probability selects a label, while the probability values require separate checks before interpreting them as trustworthy numerical confidence.

35. What is C=10?

C controls inverse regularization strength in the scikit-learn logistic model. Larger C weakens the penalty, allowing a more flexible fit; it does not mean a penalty coefficient of ten. C=10 was retained because of the joint development score, although C=1 was slightly stronger as an individual classifier on one tuning comparison.

36. Why is `max_iter=1000` not a 1,000-epoch training schedule?

It is the optimization iteration limit. The solver may converge earlier under its stopping criterion. Solver iterations are not automatically equivalent to passes over data under every algorithm. A convergence warning would need investigation; increasing a limit is useful only when optimization has actually stopped too early.

37. What does fitting k-NN do?

It mainly stores the fitting vectors and their labels for later neighbor queries. Most prediction work occurs when distances to new inputs are evaluated. Thus a short fitting time alone does not establish a fast complete notebook. The reported full-run timing also includes parsing, plotting, queries, and confidence calculations.

38. How does distance-weighted k-NN vote?

It finds the selected k nearest fitting vectors and weights labels inversely by distance. Class probabilities are normalized weighted label totals. Exact zero-distance matches receive special treatment in scikit-learn rather than an ordinary division by zero. A small distance can make one or a few neighbors dominate.

39. Why 15 neighbors rather than one?

A larger neighborhood smooths noisy individual examples, while an excessively large one can blur class boundaries and favor common industries. Development compared 5, 15, and 30 neighbors. Fifteen was retained for the selected ensemble. The choice is empirical rather than a universal rule about text embeddings.

40. Why does dimensionality matter for neighbors?

Distance can become less informative when many coordinates contribute weakly or noisily. PCA removes some variation and lowers query cost. It can also discard useful class information, so reduction must be checked empirically. The full-embedding neighbor alternative was actually compared rather than dismissed on dimensionality alone.

41. Why can an ensemble help when LR is stronger alone?

Overall strength and error complementarity are different properties. A somewhat weaker classifier can still correct some mistakes of the stronger one. The required error cross-table shows both shared and model-specific failures. The chosen vote was assessed jointly; it does not follow from assigning the larger weight to the stronger individual model.

42. How do you keep probability columns aligned?

Both classifiers fit the same encoded targets, and their class arrays were checked to match. Column j therefore refers to the same industry in each probability matrix. Adding unaligned columns would mix different labels and silently corrupt the ensemble, even if every row still summed to one.

43. Are the different “weights” in the project the same?

No. Neighbor weights are inverse distances within k-NN. Ensemble weights combine model outputs. Weighted F1 uses true validation class supports. Confidence is a correctness probability and is not a sample weight in F1. Confusing these four roles would give the wrong explanation of the official metric.

Evaluation and the official metric

44. What is precision?

For an industry, precision is TP/(TP+FP): among predictions of that industry, how many are correct? False positives come from other true industries predicted as this one. Precision alone does not measure how many members of the industry were found. The guide works through an actual class confusion count.

45. What is recall?

Recall is TP/(TP+FN): among actual members of an industry, how many receive the correct label? In a row-normalized confusion matrix, the diagonal entry is that class’s recall. This differs from column normalization, which would answer a precision-oriented question.

46. Why use the harmonic mean for F1?

F1 equals 2PR/(P+R), or 2TP/(2TP+FP+FN). The harmonic mean penalizes a poor value in either precision or recall. It is not simply their arithmetic average. Cases with undefined class ratios are handled by the report’s explicit zero-division convention rather than allowed to become NaN.

47. How is weighted F1 different from macro F1?

Weighted F1 averages class F1 values using each class’s true validation support divided by the total. Macro F1 gives every class weight 1/K. With an imbalanced dataset, high common-class performance can dominate weighted F1, so macro and per-industry results provide useful context.

48. Why is weighted F1 not identical to accuracy?

Accuracy is the total correct count divided by the number of records. F1 first combines class-specific precision and recall nonlinearly, then aggregates. Two models can have similar accuracy but different classwise error patterns and weighted F1. The notebook reports both rather than relabeling accuracy as an F-measure.

With accurate posterior probabilities and equal error costs, selecting the largest probability maximizes expected per-record accuracy. It does not directly optimize the non-additive dataset F1. The retained configuration is therefore compared empirically using the specified official score, rather than claimed to be a mathematical optimizer of weighted F1.

49. What exactly is the correctness Brier score?

Let z be one for a correct selected label and zero for an error. Brier is the mean of (confidence−z) squared. It evaluates the single returned correctness probability, not the whole 25-class probability vector. Smaller is better. A wrong prediction at confidence 0.9 contributes 0.81; a correct one contributes 0.01.

50. Why does Brier reward honest probabilities?

If true correctness probability is p, expected squared error is $(c-p)^2+p(1-p)$. This is minimized at reported confidence c=p. Perfect calibration does not imply zero Brier when outcomes remain uncertain. Confidence zero on every mistake would require knowing the unknown outcome in advance.

51. How is the final course score calculated?

Multiply support-weighted classification F1 by one minus the correctness Brier. Confidence does not alter confusion counts, weight F1 samples, or remove low-confidence cases. The implementation matches the current skeleton's mathematical definition and checks matching lengths and confidence bounds.

52. Why is the earlier 85% target not used?

The professor explicitly withdrew it after replacing the previous scoring method. The new method generally produces lower numbers. About 0.66 for plain LR on the private test is orientation, not a locally guaranteed cutoff. Final bonus thresholds will be set after the private evaluations.

53. Can you prove this submission will get the bonus?

No. The final threshold and private labels are unavailable. The defensible evidence is the same-split local baseline comparison, calibration behavior, resource measurements, and stated limitations. A local result above the indicative number cannot by itself certify an unknown private score or bonus award.

54. Why can't confidence alone push the current score arbitrarily high?

One minus Brier is at most one, so the official score cannot exceed weighted F1 for fixed predicted labels. Improving confidence reduces a penalty; it cannot repair the classification F1 term. The guide shows the current numerical ceiling separately from the observed score.

55. Does the result clearly beat LR on every dataset?

No. It is higher on this local split. The guide also reports an approximate paired bootstrap interval rather than treating a positive point difference as a universal improvement. Model selection, repeated holdout inspection, correlated companies, and private-test shift limit any broad significance claim.

56. What does the bootstrap calculation estimate?

It repeatedly resamples validation groups sharing the same exact stored embedding string, keeping methods paired on the same selected companies. It describes variability conditional on the fitted models under approximate group-sampling assumptions. It does not retrain models or account for every dependency and development decision.

Financial evidence and correctness confidence

57. Why are financial profiles based on unique fitting IDs?

A company should not count twice merely because the text table contains two records with its ID. The financial reference data drop repeated IDs after the company-level financial join. Distinct text embeddings may still teach the text classifiers, while one company supplies one observation to each financial reference sample.

58. What is an empirical mid-rank?

For value x, L counts reference values strictly below x, and R counts values below or equal to x. The chosen rank fraction is u=(L+R)/(2n). If x is tied with reference values, half the tied mass lies on each side. Without ties L=R, so the formula reduces to L/n.

59. Why use sorted references and `searchsorted`?

Sorting is done once on fitting data. Binary insertion-point searches then count below and below-or-equal observations efficiently. The two side options implement the tie convention explicitly. Computing a full comparison against every reference value for every new company would be more repetitive and costly.

60. What does 2 min(u,1−u) mean?

It measures central rank relative to a predicted industry's financial distribution: near one around its median and near zero in either tail. It stays in [0,1], so huge finite observations cannot produce unbounded numerical contributions. It is a supporting feature, rather than a direct probability of correct classification.

61. Is that value a formal p-value?

No. A p-value requires a specified null model, test statistic, sampling design, and calibration under that null. This project uses an empirical central-rank measure, which is permitted by the assignment's "other statistical measures" option. Calling it a p-value or a posterior industry probability would overstate its interpretation.

62. Does a middle rank guarantee high density?

No. A value between two separated modes can have a middle rank while lying in a low-density region. Rank compatibility measures position, not local probability density. This limitation is one reason it is supplied to a fitted correctness model as weak evidence rather than substituted for a class likelihood.

63. Why not apply a normal z-test to every ratio?

The observed ratios are skewed, heavy-tailed, sometimes negative, and often missing. A Gaussian individual-value assumption would need justification. Rank evidence avoids estimating an outlier-sensitive mean and standard deviation for every industry. It still has sampling limitations, particularly for small reference samples.

64. Why not divide an individual financial deviation by s/sqrt(n)?

That is the estimated standard error of a sample mean, not the scale of an individual observation. Doing so would make a typical individual value appear more extreme merely because the industry reference is large. Individual-value prediction intervals require their own formulas and assumptions; this notebook makes no such normal-test claim.

65. Why average only the observed ratios?

An absent ratio supplies no evidence about its financial rank. Averaging observed contributions avoids interpreting absence as a zero-valued financial ratio. A second feature records observed-count divided by three, so identical compatibility values with different information coverage remain distinguishable.

66. Does compatibility 0.5 mean a 50% correctness probability?

No. It is only the numerical placeholder used when all financial ratios are unavailable. Availability is then zero, allowing the correctness model to recognize that case. The final confidence is obtained from the fitted binary model using all features, rather than copied from that placeholder.

67. Which industry's reference is used on an unlabeled company?

The ensemble's predicted industry. It is decoded from the selected class code and used to choose the reference samples. The company's true industry is neither required nor read by the inference function. Supplying a deliberately wrong industry column was checked not to change its predictions.

68. What are the six confidence inputs?

The log-odds of the largest ensemble vote, the difference between the largest two votes, LR/k-NN label agreement, log-one-plus nearest distance, financial compatibility, and financial availability. They combine class ambiguity, model disagreement, proximity to fitting examples, and limited financial evidence.

69. Why transform the largest vote to log-odds?

A linear binary logistic model can then use a probability-relevant scale rather than fit a straight coefficient only over [0,1]. The vote is clipped to $[10^{-6}, 1-10^{-6}]$ solely to avoid infinite logarithms. This transformation does not itself calibrate confidence; its coefficients are learned on held-out records.

70. Why include nearest-neighbor distance separately?

The largest vote describes label concentration but does not fully express whether the query lies close to fitting representations. Distance supplies an additional proximity signal, including near-duplicate inputs. It is a feature, not a guarantee of novelty or correctness, and its benefit was checked during development.

71. Why use `log1p(distance)`?

It is a defined, smooth transformation at distance zero and compresses larger distances. There is no arbitrary additive constant inside an ordinary logarithm. Its value is later standardized with the other confidence inputs. It does not change which neighbors the industry classifier selected.

72. What exactly is learned during correctness calibration?

The binary target is whether each held-out ensemble label matches its known industry. A standardized logistic model learns coefficients and an intercept mapping the six evidence features to that binary outcome. The industry classifiers remain fixed, so calibration changes confidence without changing the selected label.

73. Why use `StandardScaler` inside a pipeline?

The six inputs have different units and spreads. Standardizing makes regularization less dependent on arbitrary numerical scales. The pipeline fits means and population standard deviations on calibration features only, then reuses those values at prediction. Fitting the scaler on validation would violate that separation.

74. Can you explain a confidence numerically?

Yes. Subtract each calibration-feature mean, divide by its learned scale, multiply by its coefficient, sum the contributions and intercept, then apply the sigmoid. The worked cases show those actual numbers, their financial rank calculations, the selected votes, and the resulting squared error.

75. Are model coefficients causal effects?

No. They are conditional associations under this selected sample, feature representation, and regularization. Correlated features can alter magnitudes or signs. A positive availability coefficient does not prove that obtaining another report would causally make a company's label correct, nor that the same association will hold privately.

76. Why fit a text-only calibrator too?

It is a controlled ablation: the same selected labels, same calibration records, and same model family use only the first four inputs. Comparing it with financial calibration shows the incremental contribution of the last two features. It avoids attributing every calibration improvement to financial evidence.

77. What did the extra financial-classifier check establish?

A small additional financial classifier was evaluated during development, along with a small nonlinear correctness model. Its best incremental tuning gain was too small to justify retaining another algorithm in the compact submission. Unretained checks are described as development evidence, not claimed as part of the deployed notebook.

Reliability, operation, and reproducibility

78. How should I read the reliability diagram?

Each occupied bin contributes a point whose x-coordinate is mean predicted confidence and y-coordinate is observed accuracy. A point near the diagonal indicates agreement at that resolution. Counts matter: a discrepancy in a bin with eleven cases is much less stable than one with more than a thousand.

79. Does a diagonal reliability plot prove perfect calibration?

No. Binning can conceal within-bin variation, and finite samples are noisy. Different bins can give different visual impressions. The diagram complements the continuous Brier calculation rather than replacing it. Calibration is an empirical property of a population and can change under distribution shift.

80. Would ROC AUC replace Brier?

No. AUC measures ranking discrimination across thresholds, while Brier evaluates numerical probabilities against outcomes. A monotonic transformation can preserve ranking while badly damaging calibration. The assignment specifies the confidence score, so this project focuses on that metric and the requested confidence-versus-accuracy analysis.

81. Why flag confidence below 0.8?

It is a fixed illustrative review policy, not an instructor-mandated threshold or a value optimized on validation. The notebook reports workload, flagged accuracy, and errors captured so its practical cost is visible. A real deployment would choose a policy using review capacity and explicit costs on separate development data.

82. Does removing flagged cases improve the official score?

The reported official score includes every validation case. Computing it only on retained easy cases would change the evaluation population. High retained accuracy is useful operational information, but it is not the course score and does not establish that a bonus threshold has been reached.

83. What happens when financial information is absent at inference?

Missing columns are introduced as NaN by `reindex`, and missing or infinite values contribute no financial rank evidence. The classifiers still use the required embedding. Availability distinguishes limited evidence, and the learned confidence model returns the final probability. The interface was tested with absent, NaN, and infinite financial inputs.

84. What must `predict_industry` return?

Two one-dimensional arrays of equal length, in the input-row order: predicted industry names and confidence values in [0,1]. It does not return a list of 25 probabilities in place of correctness confidence. A DataFrame may retain any external index; the function's contract concerns positional row order.

85. Can you score returned names directly against integer-coded y_val?

No. Both sides must use the same representation. Compare names with the true industry-name Series, or transform returned names using the fitted label encoder before comparing to y_val. A consistent bijective relabeling leaves the score unchanged; mixing strings and codes is not a valid evaluation.

86. What happens if an entirely new industry appears privately?

This is a closed-set classifier over the fitting labels. It cannot invent a reliable new class. Low confidence can support review but does not solve unknown-class recognition. All known industries were

present in fitting here; a genuine new-class setting would require a different task definition and validation design.

87. What establishes the runtime requirement?

The notebook was executed completely in a fresh kernel with measured elapsed time and a conservative process-memory upper bound. Those values show the current cloud run is comfortably below 45 minutes and 16 GiB. They do not certify every mid-tier laptop, so the guide gives the exact reproduction procedure and hardware caveat.

88. Where does most memory come from?

The large string-valued input table and temporary parsed coordinate lists can occupy more memory than the final float32 matrix. The retained PCA vectors and small correctness model are comparatively modest. Process measurements account for native numerical allocations better than counting Python source lines or only measuring the array dimensions.

89. Why pin package versions?

Estimator behavior and APIs can change between versions. The requirements file fixes direct dependencies used for the supplied implementation and Jupyter execution. This improves reproducibility but is not a complete operating-system or transitive-dependency lock. Python version, numerical backends, and near-tied floating-point decisions can still matter.

90. How do you reproduce the notebook from scratch?

Create a Python 3.12 virtual environment, install the pinned requirements, place the two supplied pickle files under data beside the notebook, start Jupyter, and run all cells in order. The separate guide gives the commands. No saved weights, external API key, or hidden experiment cache is needed by the submitted notebook.

91. Why keep development helpers out of the submission?

The final notebook must reproduce the retained method, not carry a framework for every discarded idea. Development choices and measured alternatives are documented in the guide. Excluding helper scripts, dataset archives, caches, and experimental artifacts keeps the delivered project small while preserving the explanation of its decisions.

92. What can still go wrong on private data?

Industry frequencies, description novelty, accounting coverage, financial scales, or error relationships can differ. Confidence learned locally may then become miscalibrated. The model also inherits limitations of the supplied embeddings and labels. These are concrete transfer limits rather than reasons to assert an unmeasured private score.

93. How is AI assistance represented?

The notebook acknowledges assistance in development and review. That does not replace the student's responsibility to understand the interface, formulas, splits, and evidence. The numbered comments and this guide support explanation of the actual code rather than a claim that the work was produced without assistance.

94. What is the strongest supported conclusion?

The retained model executes, satisfies the required output contract, and has a measured local comparison against plain LR using the correct metric. It incorporates financial statistical evidence and reports calibration and review behavior. The exact results, uncertainty interval, and private-test limitation are given together; a universal improvement is not claimed.

95. How would you improve it next if more complexity were allowed?

First obtain a genuinely new evaluation set or use a carefully grouped repeated evaluation, then investigate the largest remaining confusion pairs and financial evidence quality. Further representation or model changes should be justified by separate development gains and resource measurements. The compact submitted notebook intentionally retains only the selected method.

Reproduction and development evidence

Clean execution

The submission needs only the notebook and the two course input files at runtime. The PDF explains the implementation, and requirements.txt fixes direct package versions. Use Python 3.12 and a virtual environment:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m jupyter lab
```

On Windows, activate `.venv\Scripts\activate` instead. Place the two input pickles in `data/` beside the notebook. Open the notebook and use Restart Kernel and Run All. The final function `predict_industry(companies)` uses the fitted objects. If finances arrive separately, aggregate and join by ID as in block 02.

The notebook depends on no model cache, experiment script, external text API, private validation labels, or saved weights. Running the fixed settings is reproducible without rerunning every development trial.

What the runtime measurement covers

The verified run used Python 3.12.14 on Linux x86-64 in the cloud, with two numerical-library threads and two requested neighbor jobs. The host exposed five logical CPUs and about 33.3 GiB RAM. The memory report is a conservative sum of the runner's and child kernel's separately measured resident-memory peaks, not proof that both peaks occurred simultaneously. It is an upper bound for these two processes rather than an exact synchronized peak.

The full-run timer includes kernel startup, data parsing, transforms, classifier fitting, distance queries, tables, figures, confidence fitting, and separate real-data interface checks. The final notebook timer starts after imports and stops before those separate checks. Both describe this machine; no Intel, AMD, or Apple laptop was available for direct certification. The 45-minute requirement applies to the intended hardware, so a local run before submission remains the direct way to verify that particular machine.

What was checked

- Every final code block executed in order in a fresh Jupyter kernel.
- Company-ID sets are disjoint; all fitting industry labels are known.
- Embeddings have 768 finite coordinates and consistent float32 arrays.
- Financial reference data contain unique fitting IDs.
- LR and k-NN probability columns correspond to the same class order.
- Prediction and confidence are one-dimensional vectors with matching length.
- Confidence is finite and bounded; reversing input reverses both outputs.
- Supplying a false industry field changes neither returned output.
- Absent, NaN, and infinite financial inputs have the same no-evidence behavior.
- Perfect-score and invalid-confidence edge cases match the specified metric.
- Synthetic data were used only for preliminary functional checks. All results reported as project performance come from the supplied real company records.

Bounded development comparisons

The main small comparison covered two LR C values (1, 10), two neighbor representations (PCA128, full768), three k values (5, 15, 30), five vote weights (0, .25, .5, .75, 1), and two confidence representations. It produced 120 tuning comparisons, all using fitting-only industry models and separate 1,427-record calibration and 1,427-record tuning halves. This is development evidence, not a claim

that 120 candidates are 120 independent statistical tests or that the best tuning score is an unbiased test estimate.

LR C	Neighbor representation	k	LR weight	Confidence	Tuning score
10	PCA128	15	0.25	logit_distance	0.690528
1	Full768	5	0.5	logit_distance	0.689540
10	PCA128	15	0.25	legacy	0.689507
1	Full768	5	0.5	legacy	0.689245
10	Full768	15	0.5	logit_distance	0.688887
10	Full768	5	0.5	logit_distance	0.688206

The chosen 25/75 ensemble was stronger jointly than assigning weight only from the individual models' F1 rankings. The added log-odds/distance representation slightly improved tuning confidence quality. Standardization makes regularized confidence fitting more appropriate to the mixed feature scales.

An additional small histogram-boosted classifier used only the three financial ratios. A 5% vote raised tuning score from 0.690528 to 0.691307, less than 0.001, while larger weights were weaker. A small boosted correctness model was also weaker than logistic correctness calibration in the checked configuration. One 128-hidden-unit MLP with 101,657 parameters was tested as a development alternative: its checked mixtures were all weaker on the same tuning score. These algorithms are omitted from the submitted notebook. No result from an uncompleted trial is claimed.

The final confidence model is refitted on the full calibration pool after these settings are fixed. Its validation result is not the tuning result. The small extra financial-classifier gain did not justify another deployed algorithm given the explicit preference for a compact project.

Approximate uncertainty check

The paired bootstrap used **2728 exact validation embedding-string groups**, 2,000 replicates, and seed 42. The observed score difference is 0.005444, with approximate 95% interval [-0.004145, 0.015126].

Groups were sampled with replacement; all records in a sampled group received the same multiplicity, and both methods used identical resampling weights. F1 and Brier were recomputed for each weighted resample. This is conditional on the fitted models and approximate group independence. It excludes training variation, model selection, repeated holdout inspection, residual relationships across groups, and private-test shift. It therefore does not establish an unconditional significance result or certify bonus points.

Complete per-industry results

Fitting counts describe classifier training, while validation support is the denominator for held-out class recall and weighted-F1 aggregation. F1 entries below are fractions.

Industry	Fit	Val	LR F1	k-NN F1
Automobiles & Components	515	60	0.7049	0.7287
Banks	764	106	0.9717	0.9279
Capital Goods	2516	308	0.6474	0.6328
Commercial & Professional Services	666	82	0.3759	0.3363
Consumer Discretionary Distribution & Retail	551	57	0.5660	0.5586
Consumer Durables & Apparel	921	128	0.7549	0.7229
Consumer Services	546	58	0.7593	0.7895
Consumer Staples Distribution & Retail	151	18	0.4615	0.4286
Energy	1071	139	0.6950	0.6967
Equity Real Estate Investment Trusts (REITs)	348	51	0.8298	0.7312
Financial Services	1557	191	0.7646	0.7321
Food, Beverage & Tobacco	904	102	0.8426	0.8426
Health Care Equipment & Services	819	104	0.7633	0.7736
Household & Personal Products	189	19	0.4444	0.4651
Insurance	234	24	0.7826	0.8235
Materials	6797	824	0.9243	0.9248
Media & Entertainment	752	80	0.7595	0.7778
Pharmaceuticals, Biotechnology & Life Sciences	1563	204	0.8604	0.8610
Real Estate Management & Development	734	92	0.7363	0.7047
Semiconductors & Semiconductor Equipment	497	63	0.6949	0.6838
Software & Services	1220	167	0.6941	0.7040
Technology Hardware & Equipment	1237	177	0.6627	0.6941
Telecommunication Services	181	22	0.6667	0.7500
Transportation	480	49	0.7879	0.7573
Utilities	468	46	0.7872	0.7143

Financial reference observation counts

These are usable observed values from unique fitting company IDs after aggregation and infinity handling. They give the actual n used in the rank calculation for each ratio and industry.

Industry	Net margin	EBITDA margin	Asset turnover
Automobiles & Components	475	133	479
Banks	710	209	709
Capital Goods	2276	727	2305
Commercial & Professional Services	532	290	551
Consumer Discretionary Distribution & Retail	473	277	474
Consumer Durables & Apparel	832	272	833
Consumer Services	469	260	477
Consumer Staples Distribution & Retail	140	66	143
Energy	795	338	878
Equity Real Estate Investment Trusts (REITs)	323	221	325
Financial Services	1115	639	1293
Food, Beverage & Tobacco	782	310	790
Health Care Equipment & Services	678	411	708
Household & Personal Products	161	78	162
Insurance	211	121	212
Materials	5438	732	6193
Media & Entertainment	620	324	624
Pharmaceuticals, Biotechnology & Life Sciences	1074	572	1372
Real Estate Management & Development	636	253	643
Semiconductors & Semiconductor Equipment	457	102	464
Software & Services	1012	543	1034
Technology Hardware & Equipment	1143	308	1151
Telecommunication Services	154	87	162
Transportation	434	184	438
Utilities	414	193	424

Complete annotated code

Code block 01

```
import ast  # [01.01] Parse the supplied embedding strings as literal Python values.
from time import perf_counter  # [01.02] Measure elapsed time with a monotonic clock.
import numpy as np  # [01.03] Use arrays and vectorized numerical operations.
import pandas as pd  # [01.04] Load, join, group, and summarize tables.
import matplotlib.pyplot as plt  # [01.05] Create and label figures.
import seaborn as sns  # [01.06] Draw readable statistical plots.
from IPython.display import display  # [01.07] Display tables inside Jupyter.
from sklearn.decomposition import PCA  # [01.08] Import the principal-component transformation.
# [01.09] Import the classifier used for industries and correctness confidence.
from sklearn.linear_model import LogisticRegression
# [01.10] Import the required classification metrics.
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score
from sklearn.model_selection import GroupShuffleSplit  # [01.11] Split whole company-ID groups
↪  reproducibly.
from sklearn.neighbors import KNeighborsClassifier  # [01.12] Import the distance-weighted neighbor
↪  classifier.
# [01.13] Chain confidence scaling and logistic regression into one fitted object.
from sklearn.pipeline import make_pipeline
# [01.14] Encode industry names and scale the six confidence features.
from sklearn.preprocessing import LabelEncoder, StandardScaler
from threadpoolctl import threadpool_limits  # [01.15] Control numerical-library thread use.
threadpool_limits(2)  # [01.16] Limit numerical-library work to two threads.
started = perf_counter()  # [01.17] Start the notebook's elapsed-time measurement.
```

Code block 02

```
ratios = ['net_profit_margin', 'ebitda_margin', 'asset_turnover']  # [02.01] Name the three available
↪  financial ratios.
financial = pd.read_pickle('data/df_financials_train.pkl')  # [02.02] Load the supplied financial records.
# [02.03] Count missing and infinite values separately before cleaning.
display(pd.DataFrame({'Missing': financial[ratios].isna().sum(),
                      'Infinite': np.isinf(financial[ratios]).sum()}))
# [02.04] Treat infinities as missing and aggregate repeated financial IDs by median.
financial = financial.replace([np.inf, -np.inf], np.nan).groupby('id')[ratios].median()
# [02.05] Keep every company while joining at most one financial record per ID.
df = pd.read_pickle('data/df_train.pkl').merge(financial, on='id', how='left', validate='many_to_one')

def split(rows, size):  # [02.06] Reuse a short helper for company-group splits.
    # [02.07] Select one seeded split without separating repeated company IDs.
    left, right = next(GroupShuffleSplit(n_splits=1, test_size=size, random_state=42)
                       .split(rows, groups=rows.id))
    return rows.iloc[left], rows.iloc[right]  # [02.08] Select rows by their integer positions.

development, df_val = split(df, 0.1)  # [02.09] Reserve about ten percent of company IDs for validation.
df_fit, df_cal = split(development, 0.1)  # [02.10] Hold out part of development data for correctness
↪  calibration.
parts = [df_fit, df_cal, df_val]  # [02.11] Keep one consistent fitting/calibration/validation order.

def embeddings(rows):  # [02.12] Reuse the same embedding conversion at training and inference.
    # [02.13] Parse coordinate lists and store the numerical matrix as float32.
    return np.array(rows.business_description_embedding.apply(ast.literal_eval).tolist(),
    ↪  dtype=np.float32)

# [02.14] Convert each subset once and unpack the three matrices.
X_fit, X_cal, X_val = [embeddings(part) for part in parts]
le = LabelEncoder().fit(df_fit.industry)  # [02.15] Learn industry-to-code mapping from fitting labels.
# [02.16] Apply that same mapping to each target vector.
y_fit, y_cal, y_val = [le.transform(part.industry) for part in parts]
# [02.17] Print the actual record counts instead of assuming exact percentages.
print('Records:', len(df), '| fitting/calibration/validation:', [len(part) for part in parts])
counts = df_fit.industry.value_counts()  # [02.18] Count the records that actually teach the classifiers.
display(counts.rename('Fitting records').to_frame())  # [02.19] Display the required fitting-industry
↪  frequency table.
```

Code block 03

```
pca = PCA(n_components=128, random_state=42).fit(X_fit)  # [03.01] Learn 128 PCA axes using fitting
↪  embeddings only.
# [03.02] Project fitting embeddings with the same transformation later used for new inputs.
X_fit_pca = pca.transform(X_fit)
# [03.03] Prepare the first two coordinates and their industry labels for plotting.
plot = pd.DataFrame(X_fit_pca[:, :2], columns=['PC1', 'PC2']).assign(Industry=df_fit.industry.to_numpy())
plt.figure(figsize=(10, 6))  # [03.04] Choose a figure size that leaves room for the industry legend.
# [03.05] Color companies by industry to inspect overlap in two dimensions.
sns.scatterplot(data=plot, x='PC1', y='PC2', hue='Industry', s=7, alpha=0.3, palette='husl')
# [03.06] Place the many industry names beside the scatterplot.
plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left', fontsize=6)
# [03.07] Identify the projected fitting data in the title.
plt.title('Fitting embeddings: first two principal components')
plt.tight_layout()  # [03.08] Adjust spacing so labels remain visible.
plt.show()  # [03.09] Show the PCA scatterplot.
# [03.10] Report how much fitting variance the displayed and retained components explain.
print('Variance retained, 2 and 128 components:', pca.explained_variance_ratio_[:2].sum(),
      pca.explained_variance_ratio_.sum())
# [03.11] Fit logistic regression on the original embedding coordinates.
lr = LogisticRegression(C=10, max_iter=1000).fit(X_fit, y_fit)
# [03.12] Store the fitting PCA vectors and labels for distance-weighted 15-neighbor prediction.
knn = KNeighborsClassifier(n_neighbors=15, weights='distance', n_jobs=2).fit(X_fit_pca, y_fit)

def model_inputs(X):  # [03.13] Reuse the base-model probabilities and nearest-distance calculation.
    reduced = pca.transform(X)  # [03.14] Transform new vectors with the already fitted PCA.
    # [03.15] Return aligned LR probabilities, k-NN probabilities, and nearest distances.
    return (lr.predict_proba(X), knn.predict_proba(reduced),
            knn.kneighbors(reduced, n_neighbors=1)[0][:, 0])

# [03.16] Compute calibration and validation model inputs once.
input_cal, input_val = [model_inputs(X) for X in [X_cal, X_val]]
```

Code block 04

```
comparison = []  # [04.01] Collect comparable aggregate classifier metrics.
# [04.02] Use one analysis loop for the two required classifiers.
for name, probabilities in zip(['Logistic regression', 'k-NN'], input_val[:2]):
    prediction = probabilities.argmax(1)  # [04.03] Choose the largest class probability in each row.
    # [04.04] Record accuracy, support-weighted F1, and equally weighted macro F1.
    comparison.append([name, accuracy_score(y_val, prediction),
                       f1_score(y_val, prediction, average='weighted'),
                       f1_score(y_val, prediction, average='macro')])
    # [04.05] Build per-industry precision, recall, F1, and support results.
    report = pd.DataFrame(classification_report(y_val, prediction, target_names=le.classes_,
                                                output_dict=True, zero_division=0)).T.loc[le.classes_]
    display(report)  # [04.06] Display the per-industry report.
    # [04.07] Compare fitting class size with validation error rate descriptively.
    print(name, '| fitting size / error-rate correlation:', counts.corr(1 - report.recall))
    # [04.08] Count every true-versus-predicted industry combination.
    matrix = pd.DataFrame(confusion_matrix(y_val, prediction), index=le.classes_, columns=le.classes_)
    plt.figure(figsize=(10, 8))  # [04.09] Choose a readable confusion-matrix figure size.
    # [04.10] Divide each row by its support before plotting.
    sns.heatmap(matrix.div(matrix.sum(1), axis=0), cmap='Blues', vmin=0, vmax=1)
    plt.xlabel('Predicted industry')  # [04.11] Label the predicted-industry axis.
    plt.ylabel('True industry')  # [04.12] Label the true-industry axis.
    plt.title(name)  # [04.13] Identify which classifier produced the confusion matrix.
    plt.xticks(fontsize=6)  # [04.14] Use smaller horizontal tick labels for the 25 industry names.
    plt.yticks(fontsize=6)  # [04.15] Use smaller vertical tick labels for the same class order.
    plt.tight_layout()  # [04.16] Keep the matrix labels inside the figure layout.
    plt.show()  # [04.17] Show the row-normalized confusion plot.
    np.fill_diagonal(matrix.values, 0)  # [04.18] Exclude correct predictions from the subsequent
↪  error-pair ranking.
    # [04.19] Display the five most frequent directed industry confusions.
    display(matrix.stack().nlargest(5).rename('Errors'))
# [04.20] Display both classifiers' aggregate metrics together.
display(pd.DataFrame(comparison, columns=['Model', 'Accuracy', 'Weighted F1', 'Macro F1']))
# [04.21] Count shared and model-specific mistakes on the same companies.
display(pd.crosstab(input_val[0].argmax(1) != y_val, input_val[1].argmax(1) != y_val,
```

```
                    rownames=['LR wrong'], colnames=['k-NN wrong']))
```

Code block 05

```
reference_data = df_fit.drop_duplicates('id')  # [05.01] Use each fitting company ID once in the
↪  financial reference data.
# [05.02] Display usable financial counts and industry medians.
display(reference_data.groupby('industry')[ratios].agg(['count', 'median']))
# [05.03] Store sorted observed values for every fitting industry and ratio.
profiles = {(industry, column): np.sort(group[column].dropna().to_numpy())
            for industry, group in reference_data.groupby('industry') for column in ratios}
for column in ratios:  # [05.04] Draw one financial-distribution plot per ratio.
    plt.figure(figsize=(9, 7))  # [05.05] Choose enough height for the industry labels.
    # [05.06] Show boxplot summaries while hiding only the outlier dots.
    sns.boxplot(data=reference_data, x=column, y='industry', showfliers=False)
    # [05.07] Keep losses and extreme positive values visible with symmetric logarithmic scaling.
    plt.xscale('symlog', linthresh=0.1)
    plt.title(column.replace('_', ' '))  # [05.08] Use a readable financial-ratio title.
    plt.tight_layout()  # [05.09] Adjust the plot layout for the long industry labels.
    plt.show()  # [05.10] Show this financial-distribution plot.
```

Code block 06

```
def confidence_score(y_true, y_pred, confidence):  # [06.01] Define the current course confidence-score
↪  interface.
    # [06.02] Convert true and predicted labels to arrays for elementwise comparison.
    y_true, y_pred = np.asarray(y_true), np.asarray(y_pred)
    confidence = np.asarray(confidence, dtype=float)  # [06.03] Convert the confidence vector to
↪  floating-point values.
    # [06.04] Check that each target has a prediction and a confidence.
    assert len(y_true) == len(y_pred) == len(confidence)
    # [06.05] Reject values outside the required finite probability interval.
    assert np.all((confidence >= 0) & (confidence <= 1))
    # [06.06] Compute F1 weighted by the true class supports, without confidence weights.
    f1 = f1_score(y_true, y_pred, average='weighted')
    correct = (y_true == y_pred).astype(float)  # [06.07] Represent prediction correctness as one and
↪  error as zero.
    brier = np.mean((confidence - correct) ** 2)  # [06.08] Measure mean squared correctness-probability
↪  error.
    return f1 * (1 - brier)  # [06.09] Apply exactly weighted F1 times one minus Brier.

# [06.10] Construct confidence evidence without reading any true input industry.
def confidence_features(rows, inputs, weight):
    p_lr, p_knn, distance = inputs  # [06.11] Unpack both class-probability matrices and nearest-neighbor
↪  distances.
    # [06.12] Mix the aligned class probabilities with the selected LR weight.
    votes = weight * p_lr + (1 - weight) * p_knn
    # [06.13] Choose the class and sort votes to recover the largest two values.
    prediction, top = votes.argmax(1), np.sort(votes, axis=1)
    # [06.14] Translate predicted class codes to the financial-profile industry names.
    names = le.inverse_transform(prediction)
    financial = rows.reindex(columns=ratios)  # [06.15] Represent absent financial columns as missing
↪  values.
    # [06.16] Start each row's compatibility total and evidence count at zero.
    total, available = np.zeros(len(rows)), np.zeros(len(rows))
    # [06.17] Visit the fitting-only industry-ratio reference samples.
    for (industry, column), reference in profiles.items():
        if len(reference) == 0:  # [06.18] Check whether a reference contains any observed fitting values.
            continue  # [06.19] Skip an empty reference rather than divide by zero.
        values = financial[column].to_numpy()  # [06.20] Read this ratio in exactly the input-row order.
        mask = (names == industry) & np.isfinite(values)  # [06.21] Use finite values for the predicted
↪  industry only.
        # [06.22] Count reference observations strictly below each value.
        left = np.searchsorted(reference, values[mask], side='left')
        # [06.23] Count reference observations below or equal to each value.
        right = np.searchsorted(reference, values[mask], side='right')
       u = (left + right) / (2 * len(reference))  # [06.24] Calculate empirical mid-ranks, including ties.
       total[mask] += 2 * np.minimum(u, 1 - u)  # [06.25] Add bounded central-rank compatibility for this
↪  observed ratio.
```

```
        available[mask] += 1  # [06.26] Count this usable financial contribution.
    # [06.27] Average available evidence and use 0.5 only when none is present.
    compatibility = np.divide(total, available, out=np.full(len(rows), 0.5), where=available > 0)
    q = np.clip(top[:, -1], 1e-6, 1 - 1e-6)  # [06.28] Clip the largest vote solely to keep its log-odds
↪   finite.
    # [06.29] Combine log-odds, margin, agreement, distance, financial rank, and availability.
    features = np.column_stack([np.log(q / (1 - q)), top[:, -1] - top[:, -2],
                                p_lr.argmax(1) == p_knn.argmax(1), np.log1p(distance),
                                compatibility, available / len(ratios)])
    return prediction, features  # [06.30] Return industry codes and the six confidence-feature columns.
```

Code block 07

```
weight = 0.25  # [07.01] Use the development-selected 25 percent LR and 75 percent k-NN vote.
# [07.02] Build labels and confidence inputs on the entire held-out calibration pool.
prediction_cal, features_cal = confidence_features(df_cal, input_cal, weight)
# [07.03] Scale confidence features before regularized correctness logistic regression.
calibrator = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
# [07.04] Learn correctness, rather than industry labels, on calibration records only.
calibrator.fit(features_cal, prediction_cal == y_cal)
# [07.05] Create the same model for the text-only financial-evidence ablation.
text_calibrator = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
# [07.06] Fit that ablation using the same records and first four features.
text_calibrator.fit(features_cal[:, :4], prediction_cal == y_cal)
```

Code block 08

```
# [08.01] Build the selected ensemble's validation labels and confidence features.
prediction_val, features_val = confidence_features(df_val, input_val, weight)
# [08.02] Read the fitted probability of the correct-prediction class.
confidence_val = calibrator.predict_proba(features_val)[:, 1]
# [08.03] Compare plain classifiers, raw voting, and both calibrated confidence methods.
methods = {'Plain LR': (input_val[0].argmax(1), input_val[0].max(1)),
           'Plain k-NN': (input_val[1].argmax(1), input_val[1].max(1)),
           'Raw vote': (prediction_val, (weight * input_val[0] + (1 - weight) * input_val[1]).max(1)),
          'Text calibration': (prediction_val, text_calibrator.predict_proba(features_val[:, :4])[:, 1]),
           'Financial calibration': (prediction_val, confidence_val)}
# [08.04] Compute weighted F1, correctness Brier, and the official score for each method.
results = [[name, f1_score(y_val, prediction, average='weighted'),
            np.mean((confidence - (prediction == y_val)) ** 2), confidence_score(y_val, prediction,
↪   confidence)]
           for name, (prediction, confidence) in methods.items()]
# [08.05] Create a compact table of the actual validation results.
scores = pd.DataFrame(results, columns=['Method', 'Weighted F1', 'Brier', 'Official
↪   score']).set_index('Method')
# [08.06] Calculate the paired-split score difference from plain LR.
scores['Difference from plain LR'] = scores['Official score'] - scores.loc['Plain LR', 'Official score']
display(scores)  # [08.07] Display the baseline and ensemble results together.
# [08.08] Pair each validation confidence with its observed correctness.
outcomes = pd.DataFrame({'Confidence': confidence_val, 'Correct': prediction_val == y_val})
# [08.09] Summarize occupied fixed-width confidence bins with counts and accuracy.
reliability = outcomes.groupby(pd.cut(outcomes.Confidence, np.linspace(0, 1, 11), include_lowest=True),
↪   observed=True).agg(
    Count=('Correct', 'size'), Confidence=('Confidence', 'mean'), Accuracy=('Correct', 'mean'))
display(reliability)  # [08.10] Display the reliability-bin evidence, including sample sizes.
plt.plot([0, 1], [0, 1], '--', color='gray')  # [08.11] Draw the ideal confidence-equals-accuracy
↪   reference.
plt.plot(reliability.Confidence, reliability.Accuracy, 'o-')  # [08.12] Plot the model's observed
↪   reliability points.
plt.xlabel('Mean confidence')  # [08.13] Label the average-confidence axis.
plt.ylabel('Observed accuracy')  # [08.14] Label the observed-correctness axis.
plt.show()  # [08.15] Show the reliability plot.
review = confidence_val < 0.8  # [08.16] Flag confidence below 0.8 under a fixed illustrative review
↪   policy.
# [08.17] Compare flagged and retained groups without removing records from scoring.
display(outcomes.groupby(review).agg(Count=('Correct', 'size'), Accuracy=('Correct', 'mean')))
# [08.18] Report the review workload and how many errors it captures.
print('Review fraction:', review.mean(), '| errors captured:', np.sum(review & (prediction_val != y_val)))
```

Code block 09

```
def predict_industry(companies):  # [09.01] Expose the required prediction-and-confidence tuple interface.
    # [09.02] Reuse the parser, fitted base models, vote, and financial evidence.
    prediction, features = confidence_features(companies, model_inputs(embeddings(companies)), weight)
    # [09.03] Return industry names and correctness probabilities in input-row order.
    return le.inverse_transform(prediction), calibrator.predict_proba(features)[:, 1]

# [09.04] Demonstrate inference after removing the true industry labels.
names, confidence = predict_industry(df_val.drop(columns='industry').head(3))
# [09.05] Display three predicted industry/confidence pairs.
display(pd.DataFrame({'Prediction': names, 'Confidence': confidence}))
# [09.06] Identify exact stored embedding strings also present among fitting records.
seen = df_val.business_description_embedding.isin(df_fit.business_description_embedding)
# [09.07] Compare accuracy on seen and unseen stored representations.
display(pd.DataFrame({'Embedding seen in fitting': seen.to_numpy(), 'Correct': prediction_val == y_val})
        .groupby('Embedding seen in fitting').agg(Count=('Correct', 'size'), Accuracy=('Correct',
 ↪  'mean')))
# [09.08] Report elapsed notebook time after imports.
print('Elapsed notebook seconds:', round(perf_counter() - started, 1))
```

Sources and reading

The current-year course task and skeleton define the required methods, output contract, and official confidence-score formula. The professor's subsequent clarification withdraws the old 85% target, gives the approximate private LR baseline, requires pinned dependencies, and confirms the 45-minute/16 GB budget. The supplied main and financial pickles are the source of the numerical results. Prior-year scoring is not used.

Official implementation documentation:

- Python literal parsing
- NumPy arrays
- pandas joins
- GroupShuffleSplit
- PCA
- LogisticRegression
- KNeighborsClassifier
- StandardScaler
- Classification metrics
- Probability calibration

The general library references explain APIs and concepts. The exact official score is the course function rather than a generic multiclass Brier definition. AI assistance was used in code development and explanatory preparation.